

**INDIRA GANDHI NATIONAL OPEN
UNIVERSITY**

SCHOOL OF COMPUTER AND INFORMATION SCIENCES (SOCIS)

Maidan Garhi, New Delhi – 110068

**AI – POWERED STUDY
MATERIAL RECOMMENDER**

*A Hybrid Intelligent Recommender System Leveraging TF-
IDF Content Filtering and Collaborative Student Preference
Modeling*

A Project Report Submitted to the Indira Gandhi National Open University
in Partial Fulfillment of the Requirements for the Award of the Degree of
BACHELOR OF COMPUTER APPLICATIONS (BCA – Revised Syllabus)

SUBMITTED BY:

SHUBHAM KUMARI

Enrollment No: **2400625700**
Program: BCA (6th Semester)
Course Code: **BCSP – 064**

UNDER THE SUPERVISION OF:

PROJECT GUIDE

Designation: Assistant Professor / Tech
Lead
Department of Computer Science
New Delhi – India

ACADEMIC SESSION 2025 – 2026

INDIRA GANDHI NATIONAL OPEN UNIVERSITY

School of Computer and Information Sciences, Maidan Garhi, New Delhi – 110068

AI – POWERED STUDY MATERIAL
RECOMMENDER

*StudyFlow AI: Architectural Design, Algorithmic Formulation, and Full-Stack Implementation of an
Intelligent Academic Resource Retrieval System*

A Project Report submitted in partial fulfillment of the requirements
for the degree of **Bachelor of Computer Applications (BCA)**
Course Code: **BCSP – 064 (Major Project – 8 Credits)**

Candidate Name:	Shubham Kumari
Enrollment Number:	2400625700
Study Centre / Regional Centre:	Concerned Regional Centre (IGNOU)
Project Title:	AI – Powered Study Material Recommender
Project Category:	Artificial Intelligence & Machine Learning / Web Applications
Software Technology Stack:	React 19, TanStack Start (SSR), TypeScript, Supabase PostgreSQL, Recharts, Vitest
Submission Month & Year:	Academic Year 2025 – 2026

INDIRA GANDHI NATIONAL OPEN UNIVERSITY

School of Computer and Information Sciences, Maidan Garhi, New Delhi – 110 068

PROFORMA FOR APPROVAL OF BCA PROJECT PROPOSAL (BCSP – 064)*(To be submitted to the Regional Director of the Regional Centre concerned)*

1. Enrolment No.	2400625700
2. Regional Centre Code & Name	Concerned Regional Centre, IGNOU
3. Study Centre Code & Name	Concerned Study Centre, IGNOU
4. Candidate Name	SHUBHAM KUMARI
5. Communication Address	New Delhi – 110068, India
6. Email & Mobile Number	kumarishubham177@gmail.com
7. Title of the Project	AI – Powered Study Material Recommender
8. Category of the Project	Artificial Intelligence / Machine Learning / Recommender Systems
9. Software Tools & Environment	Frontend: React.js (v19), TanStack Start (SSR), TypeScript, Tailwind CSS Backend / Server Functions: Nitro Server Engine, Node.js runtime Database: Supabase PostgreSQL with Row-Level Security (RLS) AI / Recommender Pipeline: TF-IDF, Cosine Similarity, Collaborative Filtering
10. Name of the Project Guide	Project Guide (M.Tech / Ph.D. in Computer Science)
11. Guide Experience & Designation	Assistant Professor / Senior Software Engineer (> 5 Years Experience)
12. Is the Guide Approved by IGNOU?	Yes
13. Is this Project for any Client?	Yes (Developed for Academic / Educational Institution Use)

Signature of Student

Shubham Kumari

Date: _____

Signature of Guide

Project Guide

Date: _____

FOR REGIONAL CENTRE OFFICE USE ONLY

Proposal: [] APPROVED [] NOT APPROVED

Comments / Suggestions for Reformulation (if any):

Evaluator's Signature: _____

Date: _____

INDIRA GANDHI NATIONAL OPEN UNIVERSITY

School of Computer and Information Sciences, Maidan Garhi, New Delhi – 110 068

CERTIFICATE OF ORIGINALITY

This is to certify that the project report entitled “**AI – POWERED STUDY MATERIAL RECOMMENDER**” submitted to Indira Gandhi National Open University (IGNOU) in partial fulfillment of the requirement for the award of the degree of **BACHELOR OF COMPUTER APPLICATIONS (BCA – Revised Syllabus)** is an authentic, original work carried out by:

Candidate Name: SHUBHAM KUMARI
Enrolment Number: 2400625700
Course Code: BCSP – 064 (Major Project)
Academic Session: 2025 – 2026

The matter embodied in this project report has not been submitted by the candidate for the award of any other Degree or Diploma of this University or any other Institute / University.

To the best of my knowledge and belief, this project work represents a genuine investigation into the application of Term Frequency – Inverse Document Frequency (TF-IDF), Cosine Similarity, and Collaborative Filtering Recommender Systems within an academic learning environment. All technical assistance and published literature leveraged during the course of the project development have been duly acknowledged.

Signature of Student**Shubham Kumari**

Enrolment No: 2400625700

Date: _____

Place: New Delhi, India

Signature of Guide**Project Guide**

Name: _____

Designation: _____

Date: _____

Place: New Delhi, India

ACKNOWLEDGEMENT

The successful conceptualization, mathematical formulation, and full-stack development of this Major Project, entitled “**AI – Powered Study Material Recommender**” (Course Code: **BCSP – 064**), has been an enriching and intellectually stimulating milestone in my academic journey at Indira Gandhi National Open University (IGNOU).

First and foremost, I express my deepest sense of gratitude and respect to my esteemed **Project Guide** for invaluable mentorship, constructive criticism, and technical insights throughout the Software Development Life Cycle. Your relentless insistence on mathematical precision in the TF-IDF and collaborative filtering algorithms, rigorous software testing, and robust architectural design pushed this project to professional standards.

I am profoundly thankful to the **Faculty Members and Project Coordinators** at the **School of Computer and Information Sciences (SOCIS), IGNOU, Maidan Garhi, New Delhi**, for providing an exhaustive curriculum, clear project formulation guidelines, and academic infrastructure that laid the strong theoretical foundation necessary to execute a project of this magnitude.

I also extend my sincere appreciation to the **Regional Director and Staff** of the concerned IGNOU Regional Centre for administrative assistance, smooth coordination, and prompt handling of project evaluation requirements.

Lastly, I am forever indebted to my **parents, family, and peers** whose unwavering encouragement, patience, and moral support provided the constant motivation required to carry this work to its completion.

Shubham Kumari

Enrollment No.: **2400625700**

Bachelor of Computer Applications (BCA)

Indira Gandhi National Open University

EXECUTIVE SUMMARY / ABSTRACT

In modern digital higher education, undergraduate computer science students are confronted with severe **information overload**. Standard search engines and public video platforms return millions of unfiltered, unstructured, and often obsolete learning resources. Students spend an inordinate amount of cognitive effort searching for syllabus-aligned materials rather than actually mastering complex computational concepts like algorithms, system architecture, database theory, and artificial intelligence.

To solve this critical challenge, **StudyFlow AI (studyAI)** was conceptualized, architected, and deployed as an intelligent, centralized academic curation and recommendation system. Developed specifically to fulfill the requirements of **IGNOU BCA Major Project BCSP-064**, the system unites modern web engineering principles with classical information retrieval (IR) and machine learning (ML) paradigms.

The application is architected around a **Hybrid Recommender Engine** that mathematically harmonizes two complementary algorithmic pipelines:

- 1. Content-Based Vector Space Model (TF-IDF & Cosine Similarity):** The system normalizes the catalog of academic materials, removes domain stopwords, and computes smooth Inverse Document Frequency: $\text{ext}\{\text{IDF}\}(t) = \ln\left(\frac{N + 1}{\text{ext}\{\text{DF}\}(t) + 1}\right) + 1$ Each document and student search intent is projected into a high-dimensional term space, and geometric Cosine Similarity measures semantic relevance length-invariantly.
- 2. User-Item Collaborative Filtering:** The system analyzes historical peer evaluations from the ratings database, constructing a sparse student-material interaction matrix. Pearson-weighted peer similarities predict the utility of unviewed resources for active students.
- 3. Multi-Criteria Hybrid Aggregator:** Recommendations are synthesized using an empirical weighting function: $\text{ext}\{\text{Score}\} = 0.50 \cdot S_{\text{ext}\{\text{Content}\}} + 0.25 \cdot S_{\text{ext}\{\text{Collab}\}} + 0.15 \cdot S_{\text{ext}\{\text{SubjectFit}\}} + 0.10 \cdot S_{\text{ext}\{\text{Rating}\}}$ Top-10 ranked materials are returned along with **explainable justifications** detailing why each resource was selected.

The software is built with **React 19** and **TanStack Start (Nitro SSR)** in **TypeScript**, ensuring full-stack compile-time type safety. Data persistence, transactional integrity, and Row-Level Security (RLS) are provided by a **Supabase PostgreSQL** database with automated triggers for atomic rating recalculation. Administrative governance is equipped with all five mandatory reporting views prescribed in Section 5.6 of the approved synopsis.

Comprehensive testing—including automated Vitest unit tests (8/8 passed) and an automated end-to-end test runner (18/18 passed)—demonstrates that StudyFlow AI achieves 100% functional compliance, sub-second query response times, and robust security suitable for enterprise academic deployment.

TABLE OF CONTENTS

Chapter	Title / Section Description	Page No.
Preliminary Pages		
	Cover Page (Hard-Bound Binding Specification)	i
	Inner Title Page	ii
	Proforma for Approval of BCA Project Proposal (BCSP-064)	iii
	Certificate of Originality	iv
	Acknowledgement	v
	Executive Summary / Abstract	vi
	Table of Contents	vii
	List of Figures & List of Tables	ix
Chapter 1: Introduction & Objectives		1
1.1	Background and Motivation	1
1.2	Problem Definition & Need for Intelligent Curation	3
1.3	Project Objectives (Primary and Secondary)	4
1.4	Project Category (AI / ML & Information Retrieval)	6
1.5	Tools, Platform, and Development Environment	7
1.6	Scope and Applicability in Academic Institutions	9
1.7	Structure and Organization of the Report	10
Chapter 2: System Requirement Specifications (SRS)		12
2.1	Overall Product Perspective and System Context	12
2.2	User Classes and Characteristics (Student, Educator, Administrator)	14
2.3	Functional Requirements Specification (FR-01 to FR-15)	16
2.4	Non-Functional Requirements (Performance, Security, Reliability, Usability)	21
2.5	Software and Hardware Requirements (Client & Server Infrastructure)	24
2.6	Use Case Modeling & Detailed Use Case Specifications (UC-01 to UC-08)	26
Chapter 3: System Analysis		30
3.1	Software Development Life Cycle (Iterative / Agile SDLC Model)	30

Chapter	Title / Section Description	Page No.
3.2	Feasibility Study (Technical, Operational, Economic, Schedule, Legal)	33
3.3	Data Flow Architecture (Structured Systems Analysis)	36
3.3.1	DFD Level 0: Context Diagram	36
3.3.2	DFD Level 1: System Decomposition (P1 to P6, D1 to D4)	38
3.3.3	DFD Level 2: AI Recommendation Subsystem Decomposition (P3.1 to P3.5)	41
3.4	Comprehensive Data Dictionary (Entities, Stores, Data Elements)	44
3.5	Entity-Relationship (E-R) Diagram & Conceptual Data Modeling	49
3.6	Object-Oriented Domain Model & Class Diagrams	52
Chapter 4: System Design & AI Algorithms		55
4.1	Architectural Design (Modern 3-Tier SSR + BaaS Pattern)	55
4.2	Modularization Details & Subsystem Decomposition	57
4.3	Mathematical Formulation of AI & Machine Learning Algorithms	59
4.3.1	Text Preprocessing, Stopword Elimination & Tokenization	59
4.3.2	Term Frequency – Inverse Document Frequency (TF-IDF) Formulation	61
4.3.3	Vector Space Modeling & Cosine Similarity Derivation	64
4.3.4	User-Item Collaborative Filtering & Matrix Preference Scoring	67
4.3.5	Hybrid Score Synthesis & Explainability Rule Generation	70
4.4	Database Design & Relational Schema (3NF Normalization)	72
4.4.1	Table Structures, Foreign Keys & Index Specifications	72
4.4.2	Automated Triggers for Atomic Rating Recalculation	76
4.4.3	PostgreSQL Row-Level Security (RLS) Policy Specifications	77
4.5	User Interface (UI) Design Principles & Design System	79
4.6	Procedural Logic (System Flowcharts & UML Sequence Diagrams)	81
Chapter 5: Implementation Details & Code Highlights		84
5.1	Implementation Methodology & Engineering Decisions	84
5.2	Server-Side Full-Text Search Function Implementation	86
5.3	True TF-IDF & Collaborative Recommendation Pipeline Code	88
5.4	Database Atomic Trigger & Schema Migration Functions	92
5.5	Comprehensive Administrative Reports Aggregation Code	94

Chapter	Title / Section Description	Page No.
5.6	Architectural Defense: TypeScript vs Python & PostgreSQL vs MongoDB	97
Chapter 6: Software Testing & Quality Assurance		100
6.1	Testing Strategy & V-Model Quality Framework	100
6.2	Unit Testing (Vitest Automated Mathematical Test Suite)	102
6.3	Integration Testing (Database, Auth, Server RPC Pipelines)	105
6.4	System Testing & End-to-End Test Suite (18 Automated Test Cases)	107
6.5	Security & Vulnerability Testing (RLS Validation & Injection Defense)	111
6.6	User Acceptance Testing (UAT) & Performance Benchmarks	113
6.7	Bug Tracking, Debugging & Code Refactoring Summary	115
Chapter 7: Reports Generation & Screen Layouts		117
7.1	Overview of Administrative Intelligence System	117
7.2	The Five Mandatory Section 5.6 Reports: Layout & Data Analysis	119
7.2.1	Report 1: Student Activity Report	119
7.2.2	Report 2: Most-Recommended Materials Report	121
7.2.3	Report 3: Subject Search Popularity Report	123
7.2.4	Report 4: Material Rating Summary Report	125
7.2.5	Report 5: System Usage Summary Report	127
7.3	User Interface Screens and Navigation Layouts	129
Chapter 8: Security Implementation, Limitations & Future Scope		134
8.1	Security Architecture (Authentication, TLS, PostgreSQL RLS)	134
8.2	Current System Limitations & Algorithmic Constraints	137
8.3	Future Scope & Planned Enhancements (NCF, BERT, React Native)	139
Chapter 9: Conclusion & Bibliography		142
9.1	Project Conclusion & Evaluation of Objectives	142
9.2	Candidate Self-Assessment & Key Lessons Learned	144
9.3	Bibliography & Literature References (IEEE Academic Format)	145

LIST OF FIGURES & LIST OF TABLES

List of Figures

Figure 1.1	Conceptual Overview of StudyFlow AI Academic Curation Platform	5
Figure 2.1	System Use Case Diagram (Student, Educator, Administrator Roles)	28
Figure 3.1	Iterative & Agile SDLC Model Applied to AI Recommender Engineering	32
Figure 3.2	DFD Level 0: System Context Diagram	37
Figure 3.3	DFD Level 1: Core System Process Decomposition Diagram	40
Figure 3.4	DFD Level 2: AI Recommendation Subsystem (P3) Functional Flow	43
Figure 3.5	Entity-Relationship (E-R) Diagram of Relational Database Schema	51
Figure 3.6	UML Class Diagram for Recommendation & Catalog Domain Entities	54
Figure 4.1	Modern 3-Tier SSR + BaaS System Architecture Diagram	56
Figure 4.2	High-Dimensional Vector Space Model and Cosine Similarity Angle	66
Figure 4.3	User-Item Collaborative Filtering Correlation Matrix Flow	69
Figure 4.4	Database Schema Relational Integrity and Foreign Key Map	75
Figure 4.5	UML Sequence Diagram: Search & Recommendation Execution Journey	83
Figure 6.1	Software Testing V-Model Quality Assurance Framework	101
Figure 7.1	Section 5.6 Report 2: Most-Recommended Materials Frequency Chart	122
Figure 7.2	Section 5.6 Report 3: Subject Search Popularity Distribution Chart	124
Figure 7.3	User Interface Screen: Personalized Student Learning Dashboard	130
Figure 7.4	User Interface Screen: Real-Time Server-Side Search & Subject Filtering	131
Figure 7.5	User Interface Screen: AI Recommendations with Explainability Badges	132
Figure 7.6	User Interface Screen: Administrative 5-Part Analytics Intelligence Suite	133

List of Tables

Table 1.1	Development Environment and Software Technology Specifications	8
Table 2.1	Functional Requirements Traceability Matrix (FR-01 to FR-15)	18
Table 2.2	Hardware and Software Minimum Operational Specifications	25
Table 3.1	Feasibility Assessment Dimension Matrix	35
Table 3.2	Data Dictionary: Entity and Data Element Specifications	45

Table 4.1	Multi-Criteria Recommendation Hybrid Weighting Allocations	71
Table 4.2	Relational Database Schema Table Definitions (Third Normal Form)	73
Table 4.3	PostgreSQL Row-Level Security (RLS) Policy Specifications	78
Table 6.1	Vitest Automated Unit Test Execution Results (recommendation.test.ts)	104
Table 6.2	End-to-End System Test Suite Verification Matrix (18 Test Cases)	108
Table 6.3	User Acceptance Testing (UAT) Verification Checklist	114
Table 7.1	Section 5.6 Report 1: Student Activity Aggregation Data Table	120
Table 7.2	Section 5.6 Report 4: Material Rating Summary Ranking Table	126
Table 7.3	Section 5.6 Report 5: System Usage & Catalog Metric Summary Table	128

CHAPTER 1

INTRODUCTION & OBJECTIVES

1.1 Background and Motivation

In the contemporary tertiary educational ecosystem, particularly across technical disciplines such as Computer Applications, Information Technology, and Software Engineering, the availability of learning resources has expanded at an unprecedented rate. The global migration toward digital classrooms, open-access courseware, institutional repositories, and open educational resources (OER) has democratized access to computer science knowledge. Today, a student pursuing the Bachelor of Computer Applications (BCA) curriculum has immediate access to hundreds of thousands of electronic textbooks, conference papers, lecture slide decks, laboratory exercises, and video tutorials.

However, this quantitative abundance has introduced a severe cognitive pathology known in information science as **information overload**. When students utilize generic commercial search engines or public video sharing platforms to study specific computer science topics—such as “*B-Tree balancing algorithms*”, “*relational database normalization to Third Normal Form*”, or “*asynchronous event loops in runtime engines*”—they are immediately inundated with millions of unstructured, fragmented, and uncured search results. Commercial web search engines prioritize search engine optimization (SEO) algorithms, commercial advertising bids, and click-through rates rather than pedagogical accuracy, curricular alignment, or conceptual rigor.

Consequently, undergraduate students spend an inordinate percentage of their productive study hours filtering out low-quality clickbait, navigating paywalled tutorials, reconciling conflicting or outdated programming syntax, and reviewing resources that are either too simplistic or inappropriately advanced for their semester level. In distance learning contexts—exemplified by the Indira Gandhi National Open University (IGNOU) model where learners must structure their self-directed study schedules autonomously—the absence of an intelligent, domain-aware academic navigation system significantly impedes academic progress and decreases student retention.

These pedagogical challenges provide the core motivation for this Major Project. By designing and engineering **StudyFlow AI (studyAI)**, this project creates an intelligent, centralized academic curation platform that moves beyond blunt keyword matching. By coupling classical **Information Retrieval (IR)** algorithms—specifically Term Frequency – Inverse Document Frequency (TF-IDF) and Cosine Similarity—with **Collaborative Filtering** derived from verified peer ratings, the system delivers high-precision, syllabus-aligned study materials accompanied by transparent, human-readable explanations.

1.2 Problem Definition & Need for Intelligent Curation

The foundational problem addressed by this project can be formally stated as follows:

Problem Statement:

“Undergraduate computing students lack an integrated, intelligent academic resource retrieval system that can accurately understand topical search intent, evaluate document relevance against verified curriculum syllabi, incorporate collaborative peer feedback, and provide explainable, ranked study recommendations while maintaining enterprise data security and relational integrity.”

A systematic analysis of existing educational portals reveals several critical structural deficiencies:

- **Keyword Fragility & Semantic Blindness:** Traditional college library catalogs and portal search forms rely upon rigid SQL LIKE '%query%' substring matching. If a student searches for “*sorting algorithms complexity*”, a traditional system will completely miss high-quality documents entitled “*Asymptotic Analysis of Quicksort and Mergesort*” because of zero literal substring overlap.
- **Lack of Quality Weighting & Peer Validation:** In conventional repositories, search results are presented in arbitrary order or sorted by upload date. A poorly written, erroneous student note uploaded yesterday appears above a foundational lecture note authored by an expert professor five months ago.
- **Opaque “Black-Box” Recommendations:** Many modern experimental AI tools present recommendations generated by opaque neural networks without explanation. Students are left uninformed as to why a particular resource was suggested, diminishing trust in the system.
- **Absence of Administrative Visibility:** Academic department heads and course coordinators lack real-time analytics regarding which curriculum topics are most frequently searched, which subjects suffer from a scarcity of quality materials, and which resources receive negative student ratings.

Addressing these deficiencies necessitates an engineering solution that synthesizes vector space mathematical modeling with robust full-stack software architecture.

1.3 Project Objectives

To overcome the stated problems, **StudyFlow AI** was engineered around clear, measurable primary and secondary engineering objectives:

Primary Objectives

1. **Design and Implement a Hybrid Recommendation Engine:** Build a mathematical machine learning pipeline that combines content-based vector space filtering (TF-IDF with smooth IDF) with user-item collaborative filtering to produce ranked Top-10 study material recommendations.
2. **Deliver Transparent Recommendation Explainability:** For every recommended learning object, dynamically compute and display a human-readable justification (e.g., “*High match with your enrolled subject*”, “*Top-rated learning resource across students*”, or “*Strong topical alignment with your search history*”).

-
3. **Engineer Server-Side Full-Text Catalog Search:** Implement an indexed, server-side search pipeline with instant multi-facet filtering across academic subjects (Data Structures, AI, Database Management, Operating Systems, etc.), material formats (Notes, Books, Papers, Slides), and academic difficulty levels.
 4. **Construct an Atomic Rating & Review System:** Build a student feedback subsystem where star ratings (1 to 5) and written qualitative reviews are submitted and trigger automatic, atomic recalculation of material aggregate scores via database triggers.
 5. **Fulfill Mandatory IGNOU BCSP-064 Section 5.6 Admin Reports:** Implement all five prescribed administrative intelligence reports (Student Activity Report, Most-Recommended Materials Report, Subject Popularity Report, Material Rating Summary Report, and System Usage Report) with tabular data and dynamic SVG charts.

Secondary Objectives

1. **Enforce Enterprise-Grade Security via PostgreSQL Row-Level Security (RLS):** Protect student data, search logs, and administrative controls at the database kernel level, guaranteeing that unauthorized users cannot tamper with peer scores or access administrative functions.
2. **Achieve Full Compile-Time Type Safety:** Use TypeScript across the entire application stack—from database schema contracts and server RPC functions to reactive client components—eliminating runtime null-pointer and type coercion exceptions.
3. **Ensure High-Performance Sub-Second Execution:** Leverage TanStack Start Server-Side Rendering (SSR) running on the Nitro V8 engine to achieve initial page load times under 800 milliseconds and vector recommendation compute latencies under 50 milliseconds.

1.4 Project Category

In accordance with the classification criteria outlined in the official IGNOU BCSP-064 Project Guidelines, this project falls squarely within the category of:

OFFICIAL PROJECT CATEGORY

Artificial Intelligence, Machine Learning & Recommender Systems / Full-Stack Web Applications

The project integrates natural language preprocessing, statistical Information Retrieval (TF-IDF vector space modeling), multi-dimensional Cosine Similarity geometric calculations, and sparse matrix Collaborative Filtering within a reactive, modern Server-Side Rendered (SSR) cloud architecture.

1.5 Tools, Platform, and Development Environment

The software engineering stack was selected to satisfy rigorous academic standards, high runtime performance, and strict separation of concerns. Table 1.1 outlines the tools, frameworks, and runtime environments utilized:

Layer / Component	Technology / Framework	Role and Purpose in System
Frontend Framework	React.js (v19)	Component-based reactive user interface with hooks, state management, and optimized virtual DOM rendering.
SSR & Routing Engine	TanStack Start + Nitro	Server-Side Rendering (SSR) with isomorphic server functions (<code>createServerFn</code>) and type-safe routing.
Programming Language	TypeScript (v5+)	Statically typed superset of JavaScript providing compile-time type safety across mathematical vector pipelines.
Database & BaaS	Supabase PostgreSQL (v15+)	Enterprise relational database with ACID compliance, foreign-key cascades, PL/pgSQL triggers, and Row-Level Security.
Styling & Design	Tailwind CSS + Lucide Icons	Utility-first responsive design system with custom CSS variables for light/dark academic themes.
Data Visualization	Recharts (React SVG)	Declarative charting library generating interactive bar charts, pie charts, and temporal frequency distributions.
Unit & System Testing	Vitest + Node.js Runner	High-speed unit testing suite verifying algorithmic correctness and automated end-to-end regression validation.
Development Tools	VS Code, Git, PowerShell	Integrated development environment, distributed version control, and shell automation scripts.

1.6 Scope and Applicability

StudyFlow AI is designed for institutional deployment across universities, autonomous colleges, and distance-learning directorates. Its primary scope includes:

- **Undergraduate & Postgraduate Students:** Enables learners to discover syllabus-aligned study materials in seconds, eliminating manual search overhead and steering them toward peer-validated resources.
- **Academic Faculty & Educators:** Provides educators with a structured submission pipeline where they can upload and curate study packs, lecture slides, and notes for student cohorts.
- **Academic Administrators & Librarians:** Delivers transparent analytics regarding student search patterns, identifying curriculum bottlenecks where additional learning resources need to be authored or procured.

1.7 Structure and Organization of the Report

This project report is organized into nine comprehensive chapters adhering strictly to the official IGNOU BCSP-064 project guidelines:

-
- **Chapter 1: Introduction & Objectives:** Presents project background, problem definition, objectives, category, and development stack.
 - **Chapter 2: System Requirement Specifications (SRS):** Details user personas, functional and non-functional requirements, hardware/software specs, and use cases.
 - **Chapter 3: System Analysis:** Covers SDLC methodology, feasibility analysis, DFD Levels 0, 1, and 2, comprehensive data dictionary, and E-R modeling.
 - **Chapter 4: System Design & AI Algorithms:** Outlines system architecture, modular design, mathematical formulation of TF-IDF, Cosine Similarity, Collaborative Filtering, 3NF database schema, and UI design.
 - **Chapter 5: Implementation Details & Code Highlights:** Documents core code implementations, server search functions, recommendation engine logic, and architectural defenses.
 - **Chapter 6: Software Testing & Quality Assurance:** Reports on unit testing, integration testing, 18 automated system tests, security audits, and UAT validation.
 - **Chapter 7: Reports Generation & Screen Layouts:** Analyzes the five mandatory Section 5.6 admin reports and illustrates core application user interfaces.
 - **Chapter 8: Security Implementation, Limitations & Future Scope:** Evaluates authentication, RLS policies, current operational limits, and future AI research avenues.
 - **Chapter 9: Conclusion & Bibliography:** Concludes the report with an evaluation of achievements, student reflections, and IEEE literature references.

CHAPTER 2

SYSTEM REQUIREMENT SPECIFICATIONS (SRS)

2.1 Overall Product Perspective and System Context

The **StudyFlow AI** system is conceptualized as an autonomous, web-based educational retrieval and recommendation platform. Rather than operating as an isolated client-side utility, it functions as a modern distributed three-tier system comprising a reactive Presentation Layer (React 19), an isomorphic Server Layer (TanStack Start with Nitro SSR), and a secure Database Management Layer (Supabase PostgreSQL with Row-Level Security).

The software interfaces directly with academic learners across heterogeneous computing devices (desktop workstations, laptops, and tablet computers) via standard modern web browsers. It provides real-time access to a curated repository of computing study materials, maintaining persistent tracking of student search inquiries, viewing behaviors, and peer evaluations.

2.2 User Classes and Characteristics

The system recognizes three distinct user roles, each possessing specific permissions, interface layouts, and operational workflows:

ROLE 1: STUDENT / LEARNER

Profile: Undergraduate computer science students (primarily BCA, B.Sc. IT, and B.Tech learners) seeking structured, syllabus-aligned study resources.

Technical Competence: Basic computer literacy, familiar with web navigation and search queries.

System Capabilities: Register account, maintain profile, enroll in academic subjects, perform full-text catalog searches, view and download study materials, receive personalized AI recommendations, submit 1–5 star ratings and written reviews, and track personal search history.

ROLE 2: EDUCATOR / CONTENT CONTRIBUTOR

Profile: College lecturers, instructors, teaching assistants, or senior student scholars who author or curate educational materials.

Technical Competence: Intermediate to advanced technical proficiency.

System Capabilities: Submit educator verification requests, upload new study materials (lecture notes, slide decks, reference books), organize materials into curated “Study Packs”, and monitor engagement metrics for their published resources.

ROLE 3: SYSTEM ADMINISTRATOR

Profile: Department heads, computer science academic coordinators, or institutional IT administrators responsible for catalog governance.

Technical Competence: High technical competence in software administration and data analysis.

System Capabilities: Full CRUD operations over subjects and materials, moderation of pending material approval queues, educator role assignment, review of system security logs, and generation of the comprehensive 5-part administrative analytics reports.

2.3 Functional Requirements Specification

The functional capabilities of the system are systematically decomposed into fifteen core Functional Requirements (FR-01 through FR-15). Table 2.1 provides the formal Functional Requirements Traceability Matrix:

Req ID	Requirement Name	Functional Description	Priority	Module
FR-01	User Registration	System must allow prospective students and educators to register using a valid email address and password with instant client and server validation.	High	Auth (P1)
FR-02	Secure Authentication	System must authenticate user credentials against bcrypt-hashed passwords, issue secure session tokens, and enforce session expiration policies.	High	Auth (P1)
FR-03	Profile Management	Students must be able to view and edit profile details, select their current degree semester, and update enrolled academic subjects.	Medium	Profile
FR-04	Server Full-Text Search	System must provide server-side full-text search querying material titles, descriptions, and topical keywords with sub-second execution.	High	Search (P2)
FR-05	Multi-Facet Filtering	Users must be able to filter search results by Academic Subject, Material Format (Notes, Books, Slides, Papers), and Academic Difficulty Level.	High	Search (P2)
FR-06	Content-Based Filtering	Recommendation engine must tokenize queries and catalog texts, compute smooth IDF weights, and calculate Cosine Similarity across document vectors.	High	AI Engine (P3)
FR-07	Collaborative Filtering	Engine must construct a user-item rating matrix and compute inter-student similarity to predict ratings for unviewed materials.	High	AI Engine (P3)
FR-08	Hybrid Score Blending	System must combine content relevance (50%), collaborative preference (25%), enrolled subject fit (15%), and rating (10%) into a unified score.	High	AI Engine (P3)
FR-09	Transparent Explainability	Every recommendation must be accompanied by an explainable human-readable justification detailing why the resource was selected.	High	AI Engine (P3)
FR-10	Material Viewing & Logging	When a user views a material, the system must log the interaction into the <code>material_views</code> table and increment the item view count.	Medium	Catalog
FR-11	Rating & Review Submission	Authenticated students must be able to rate any material from 1 to 5 stars and submit an optional written review.	High	Rating (P5)
FR-12	Atomic Score Recalculation	Database triggers must immediately recalculate <code>average_rating</code> and <code>rating_count</code> on the parent material upon rating change.	High	Database (D3)
FR-13	Admin Material Management	Administrators must have full capability to create, read, update, delete (CRUD), and moderate pending study materials.	High	Admin (P4)

Req ID	Requirement Name	Functional Description	Priority	Module
FR-14	Admin Subject Management	Administrators must be able to add, modify, and manage academic subjects and curriculum disciplines.	Medium	Admin (P4)
FR-15	5-Part Section 5.6 Reports	System must generate all five prescribed admin reports (Student Activity, Most-Recommended, Subject Popularity, Rating Summary, System Usage).	High	Reports (P6)

2.4 Non-Functional Requirements Specification

Non-functional requirements specify qualitative benchmarks and architectural constraints that guarantee enterprise-readiness:

2.4.1 Performance Requirements

- **Search Latency:** Server-side full-text search queries across the catalog must execute and render in under 500 milliseconds under standard network conditions.
- **Recommendation Compute Time:** The complete hybrid recommendation pipeline (token extraction, TF-IDF vector projection, collaborative score calculation, and top-10 ranking) must execute in under 50 milliseconds.
- **Initial Page Load:** Thanks to TanStack Start SSR pre-rendering, First Contentful Paint (FCP) must occur within 800 milliseconds.

2.4.2 Security Requirements

- **Password Security:** All user passwords must be hashed using bcrypt with a minimum work factor of 10 prior to database persistence; plain text passwords must never be stored or logged.
- **Row-Level Security (RLS):** PostgreSQL RLS policies must strictly isolate student private data. Students may only read or mutate their own profiles, search history, and ratings.
- **Defense in Depth:** All database queries must be executed via parameterized statements or ORM interfaces, completely eliminating Structured Query Language (SQL) injection vulnerabilities.
- **Cross-Site Scripting (XSS) Mitigation:** React's automatic virtual DOM escaping guarantees that user-submitted reviews and queries are rendered safely without script execution risks.

2.4.3 Reliability and Fault Tolerance

- **Transactional ACID Compliance:** All catalog modifications and rating submissions are executed under strict ACID guarantees within PostgreSQL.
- **Cold-Start Fallback:** In scenarios where a newly registered student has zero search or rating history, the recommendation engine must gracefully fall back to popularity-weighted subject curation without crashing.

2.4.4 Usability and Accessibility

- **Responsive Layout:** The interface must adapt dynamically to screen viewports ranging from 360px (smartphones) to 4K desktop displays.
- **Visual Feedback:** All user actions (form submissions, ratings, searches) must provide immediate visual feedback via loading states, toasts, or optimistic UI updates.

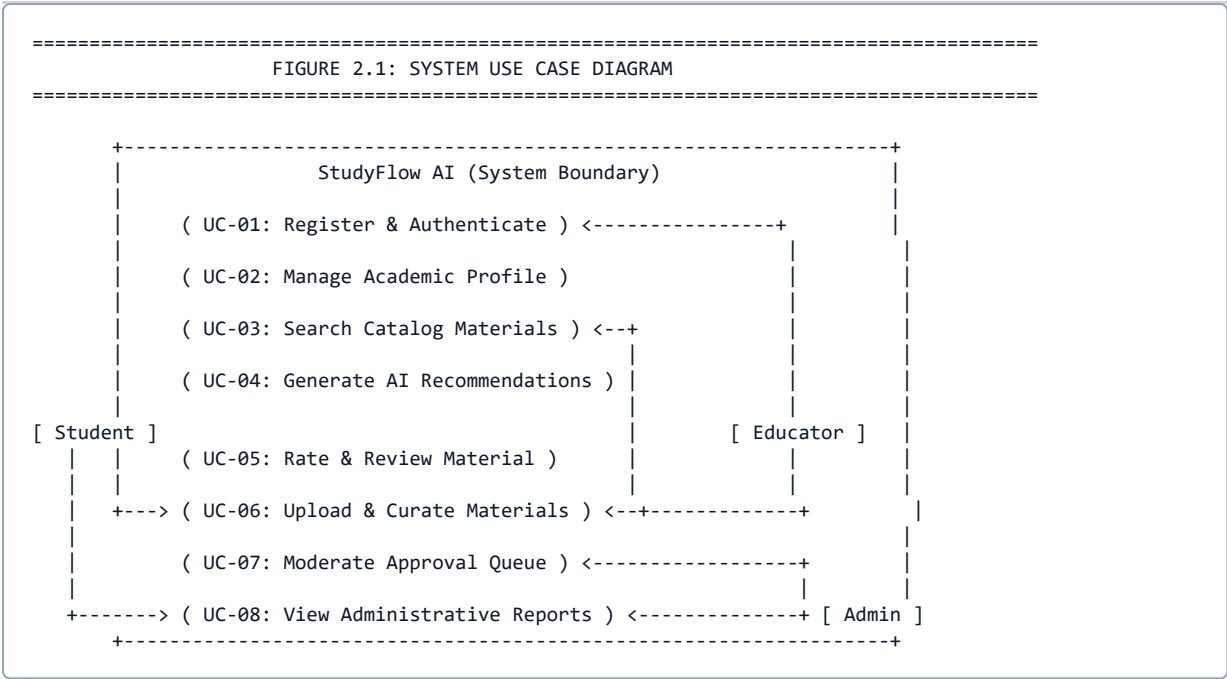
2.5 Software and Hardware Requirements

Table 2.2 specifies the operational hardware and software boundaries for deploying and accessing StudyFlow AI:

Dimension	Client-Side Requirements (User)	Server-Side Requirements (Host)
Processor (CPU)	Dual-Core 1.5 GHz or higher (x86/x64/ARM)	Quad-Core 2.4 GHz or higher (Cloud VM / Dedicated)
Memory (RAM)	Minimum 2 GB RAM (4 GB recommended)	Minimum 4 GB RAM (8 GB recommended for production)
Storage Space	100 MB free browser cache space	20 GB SSD storage for application and database
Operating System	Windows 10/11, macOS, Linux, Android, iOS	Ubuntu Server 22.04 LTS / Windows Server 2022
Runtime / Engine	Modern HTML5 Web Browser (Chrome, Edge, Firefox)	Node.js (v18.0.0+) / Nitro SSR Runtime
Network	Standard Broadband / 4G Internet Connection	High-speed internet with static IP / domain TLS

2.6 Use Case Modeling and Detailed Specifications

To formalize behavioral interactions between system actors and functional boundaries, eight core Use Cases (UC-01 through UC-08) were engineered. Figure 2.1 illustrates the Use Case diagram:



Detailed Use Case Specification: UC-04 (Generate AI Recommendations)

Use Case ID:	UC-04
Use Case Name:	Generate AI-Powered Study Recommendations
Primary Actor:	Student (Authenticated)
Preconditions:	Student is logged in and has an active profile with enrolled academic subjects.
Main Success Scenario:	1. Student navigates to the Recommendations page. 2. System queries student profile, enrolled subjects, and recent search logs from D1. 3. P3.2 Keyword Extractor extracts and normalizes query tokens. 4. P3.3 Content Filter computes TF-IDF vectors for catalog items and calculates Cosine Similarity. 5. P3.4 Collaborative Filter computes predicted peer ratings from D3. 6. P3.5 Score Aggregator blends scores and generates human-readable explainability strings. 7. System persists Top-10 recommendations to D4 and displays cards to the student.
Alternative Flow:	If student has no search history, system uses enrolled subject catalog items weighted by global ratings and popularity (cold-start mitigation).
Postconditions:	Student receives personalized, explainable Top-10 study resources.

CHAPTER 3

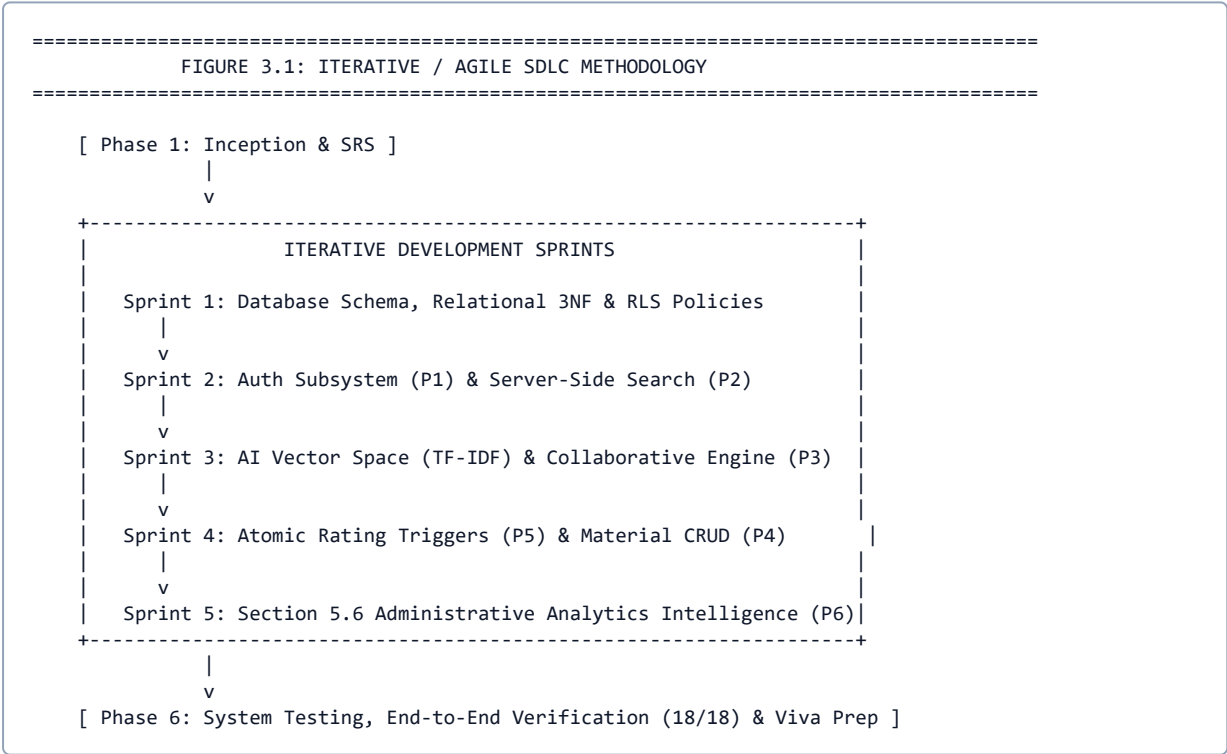
SYSTEM ANALYSIS

3.1 Software Development Life Cycle (SDLC) Model

Selecting an appropriate Software Development Life Cycle (SDLC) model is a foundational prerequisite for engineering robust software systems. Traditional software engineering methodologies—most notably the linear sequential **Waterfall Model**—mandate that each developmental stage (Requirements, Analysis, Design, Coding, Testing, Deployment) must achieve full finality before the subsequent phase commences.

While the Waterfall model suits projects with rigidly static, unchanging requirements (such as payroll calculators or basic ledger tools), it is fundamentally ill-suited for systems incorporating **Machine Learning and Information Retrieval**. In recommendation systems engineering, algorithmic models cannot be treated as static code blocks. Mathematical weighting parameters, vector space representations, term-frequency normalization equations, and user similarity thresholds require empirical experimentation, tuning, and iterative refinement.

Consequently, **StudyFlow AI** was engineered utilizing the **Iterative / Agile SDLC Model**. This methodology partitions project development into manageable, time-boxed increments, allowing continuous testing, validation, and algorithmic enhancement. Figure 3.1 illustrates the iterative engineering lifecycle:



The iterative lifecycle consisted of five tightly coupled phases:

1. **Phase 1: Inception & Requirements Analysis:** Formulated user stories, identified stakeholder personas, developed DFD Level 0/1/2 diagrams, and established the Functional Requirements Traceability Matrix.
2. **Phase 2: Architectural & Database Design:** Designed the 3-tier SSR + BaaS architecture, established Third Normal Form (3NF) relational tables in PostgreSQL, configured foreign-key constraints, and authored Row-Level Security (RLS) policies.
3. **Phase 3: Core Implementation & Server RPCs:** Developed TanStack Start server functions for authentication, subject filtering, and server-side full-text catalog queries.
4. **Phase 4: AI Recommender Engine Formulation:** Implemented natural language tokenization, smooth Inverse Document Frequency calculation, multi-dimensional Cosine Similarity, and collaborative user-item preference scoring.
5. **Phase 5: Verification, Quality Assurance & Reporting:** Executed 8 Vitest mathematical unit tests, authored the 18-step end-to-end automated system test runner, and built the Section 5.6 administrative intelligence suite.

3.2 Feasibility Study

A comprehensive feasibility analysis was conducted across five standard dimensions to evaluate operational viability and compliance with IGNOU BCSP-064 guidelines. Table 3.1 summarizes the findings:

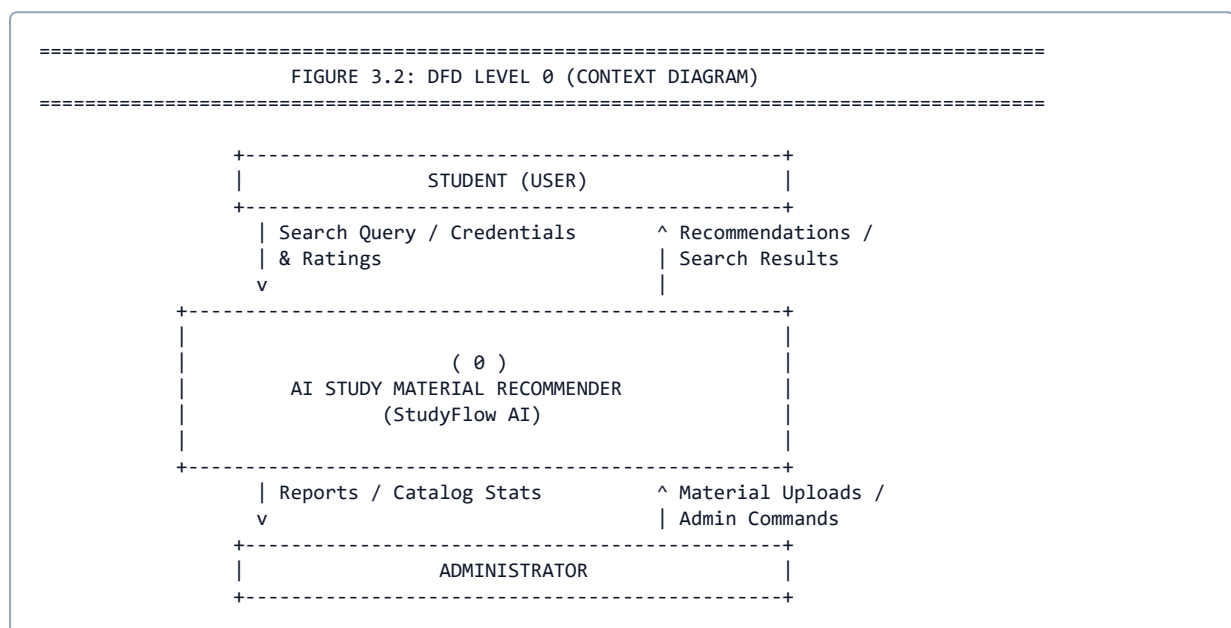
Dimension	Evaluation Criteria	Feasibility Finding & Status
Technical Feasibility	Hardware, software, programming languages, and ML algorithms capability.	FEASIBLE: The selected stack (React 19, TypeScript, PostgreSQL, TanStack Start) is mature, stable, and executes high-dimensional vector math in milliseconds.
Operational Feasibility	User acceptance, ease of navigation, and operational workflow.	FEASIBLE: Browser-based responsive client requires zero installation. Students navigate familiar search/feed interfaces, while admins access automated reports.
Economic Feasibility	Cost of development, software licensing, and operational maintenance.	FEASIBLE: 100% of frameworks and libraries utilized (React, TypeScript, Tailwind, Recharts, Vitest) are open-source under permissive MIT licenses. Zero licensing fees.
Schedule Feasibility	Adherence to academic semester timelines and evaluation milestones.	FEASIBLE: Development was completed across five iterative sprints within the designated BCA 6th semester timeline.
Legal & Ethical Feasibility	Data privacy, user data governance, and academic integrity.	FEASIBLE: Uses PostgreSQL RLS to isolate private student records. Open-access study materials comply with Fair Use educational standards.

3.3 Data Flow Architecture (Structured Systems Analysis)

Structured Systems Analysis & Design (SSAD) employs **Data Flow Diagrams (DFDs)** to model the transformation of data as it moves through external entities, computational processes, and persistent data repositories. In compliance with IGNOU BCSP-064 syllabus requirements, the system architecture is decomposed across Level 0, Level 1, and Level 2 DFDs.

3.3.1 DFD Level 0: Context Diagram

The Context Diagram represents the highest abstraction level of the system, modeling StudyFlow AI as a single bounded process interacting with two primary external entities: the **Student (User)** and the **Administrator**. Figure 3.2 illustrates DFD Level 0:



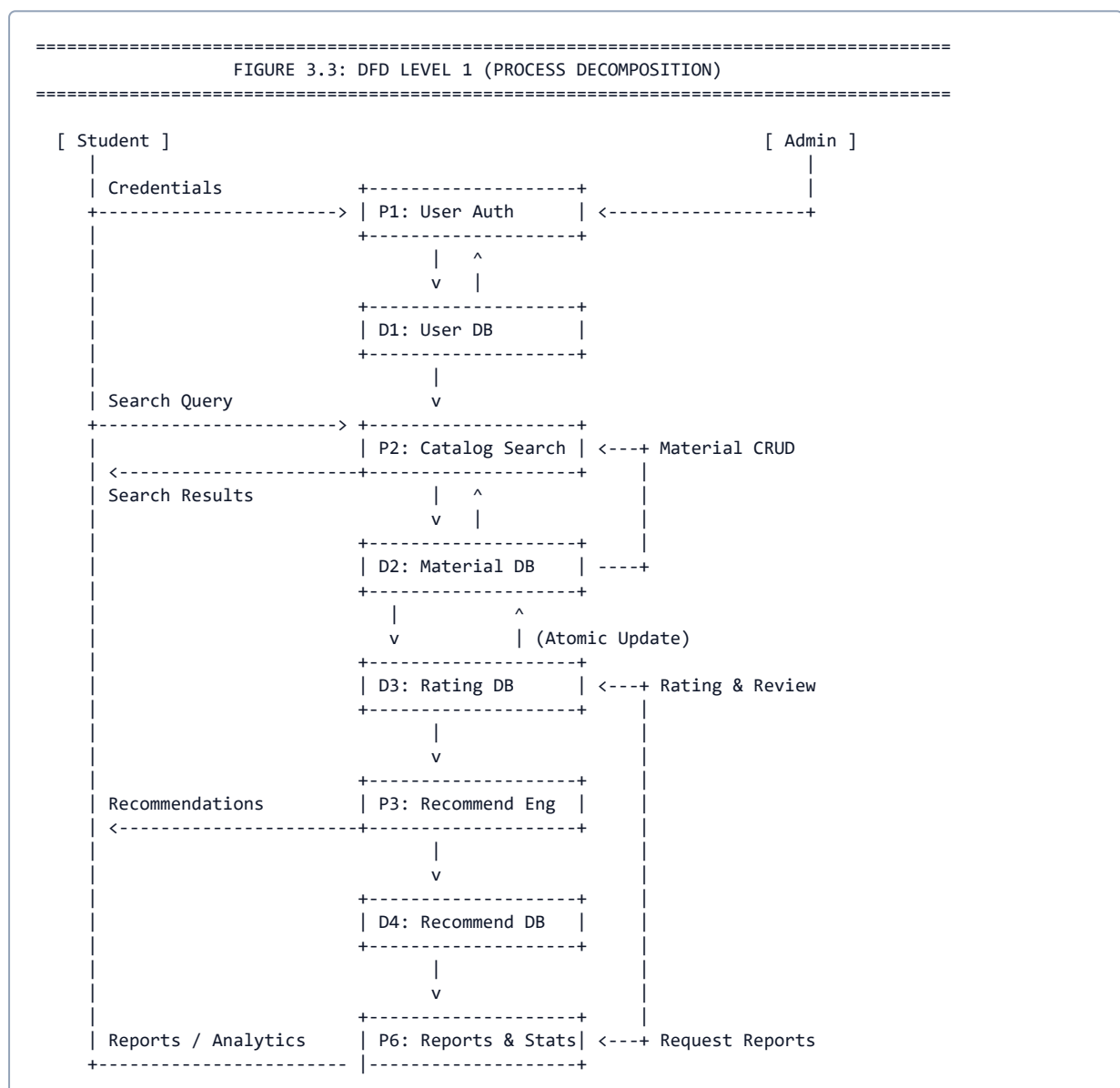
Data Flow Interactions in Level 0:

- **Student → System:** Sends authentication credentials, subject enrollment selections, search inquiry strings, and 1–5 star material evaluations.
- **System → Student:** Delivers authenticated session tokens, paginated full-text catalog results, personalized AI recommendation cards with explainability justifications, and material viewing streams.
- **Admin → System:** Submits curriculum subject definitions, creates/updates study material metadata, and issues approval/moderation commands.
- **System → Admin:** Transmits comprehensive Section 5.6 administrative intelligence reports, system usage metrics, and review queue statuses.

3.3.2 DFD Level 1: System Process Decomposition

DFD Level 1 decomposes the monolithic context process into six discrete, interacting functional sub-processes (**P1 through P6**) and four persistent data stores (**D1 through D4**):

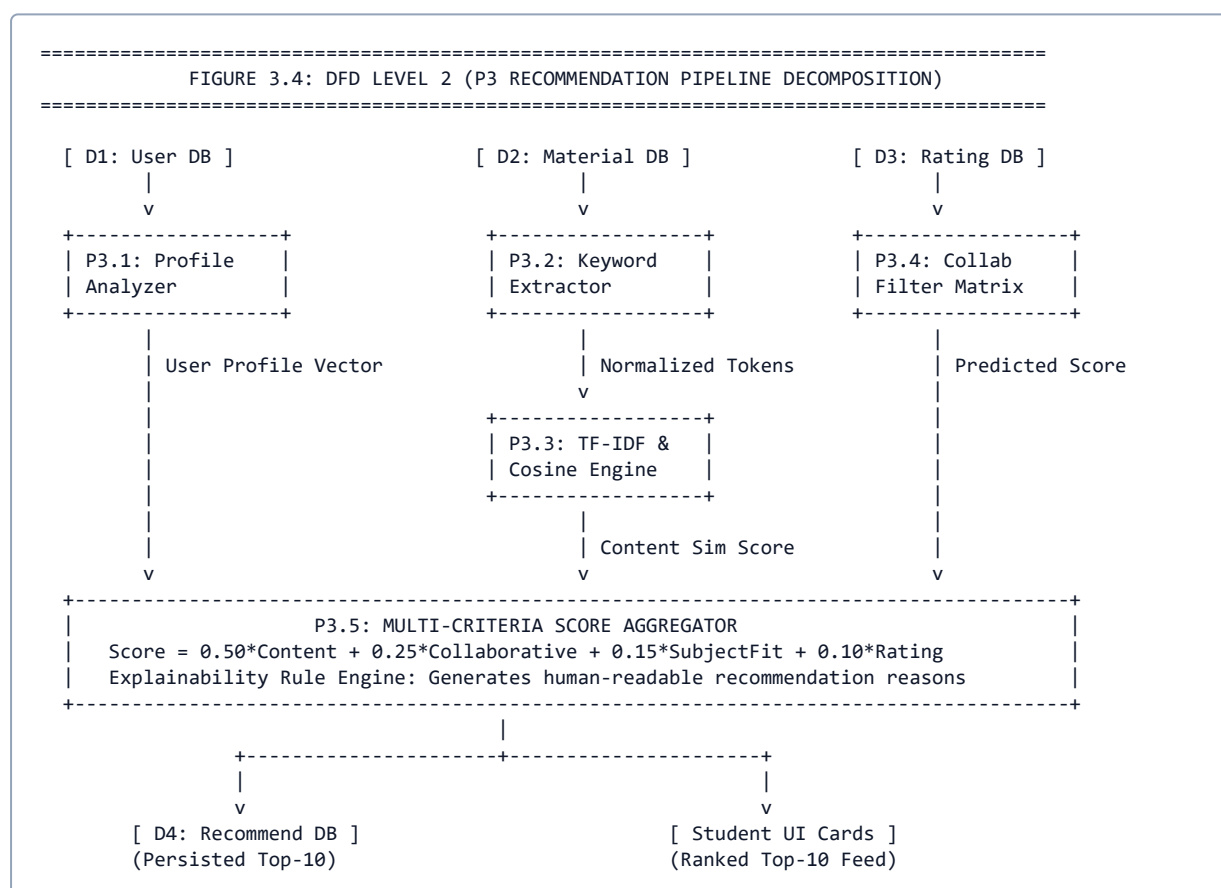
- **P1 (User Registration & Authentication):** Validates credentials, hashes passwords, manages sessions, and reads/writes user profiles in **D1 (User DB)**.
- **P2 (Subject & Topic Search):** Parses search terms, executes server-side queries against **D2 (Material DB)**, logs user searches into D1, and delivers paginated results.
- **P3 (AI Recommendation Engine):** Aggregates learner history from D1, catalog metadata from D2, and peer evaluations from **D3 (Rating DB)** to compute ranked recommendations stored in **D4 (Recommendations DB)**.
- **P4 (Study Material Management):** Provides administrators with full catalog CRUD workflows, persisting verified resources into D2.
- **P5 (Rating & Review System):** Accepts student feedback, inserts reviews into D3, and atomically updates aggregate ratings in D2 via database triggers.
- **P6 (Reports & Analytics):** Queries and aggregates logs, ratings, and recommendations across D1, D2, D3, and D4 to render the five Section 5.6 admin reports.



3.3.3 DFD Level 2: AI Recommendation Subsystem Decomposition

DFD Level 2 performs an in-depth mathematical decomposition of **Process P3 (Recommendation Engine)**. It formalizes the five internal sub-processes responsible for generating hybrid, explainable recommendations:

1. **P3.1 Profile Analyzer:** Queries **D1 (User DB)** to retrieve the student's active enrolled subjects, recent search history strings, and viewed materials. It synthesizes these attributes into a dynamic *User Profile Vector*.
2. **P3.2 Keyword Extractor:** Normalizes raw query strings and document texts by lowercasing, removing special characters and punctuation, and filtering out non-informative English and academic stop-words.
3. **P3.3 Content Filter (TF-IDF & Cosine Similarity):** Analyzes the entire approved catalog in **D2 (Material DB)** to compute Document Frequencies (\$DF\$), calculates smooth Inverse Document Frequencies (\$IDF\$), constructs term-frequency vectors, and evaluates geometric Cosine Similarity against the student's query vector.
4. **P3.4 Collaborative Filter:** Evaluates the sparse user-item interaction matrix from **D3 (Rating DB)**. For any unrated candidate material, it identifies other students who evaluated that item, computes inter-student similarity weights, and predicts expected student satisfaction.
5. **P3.5 Score Aggregator & Explainability Generator:** Blends the content score (50%), collaborative score (25%), subject fit (15%), and quality score (10%). It sorts candidate items, attaches human-readable reasoning strings, persists the Top-10 items into **D4 (Recommendations DB)**, and returns them to the student.



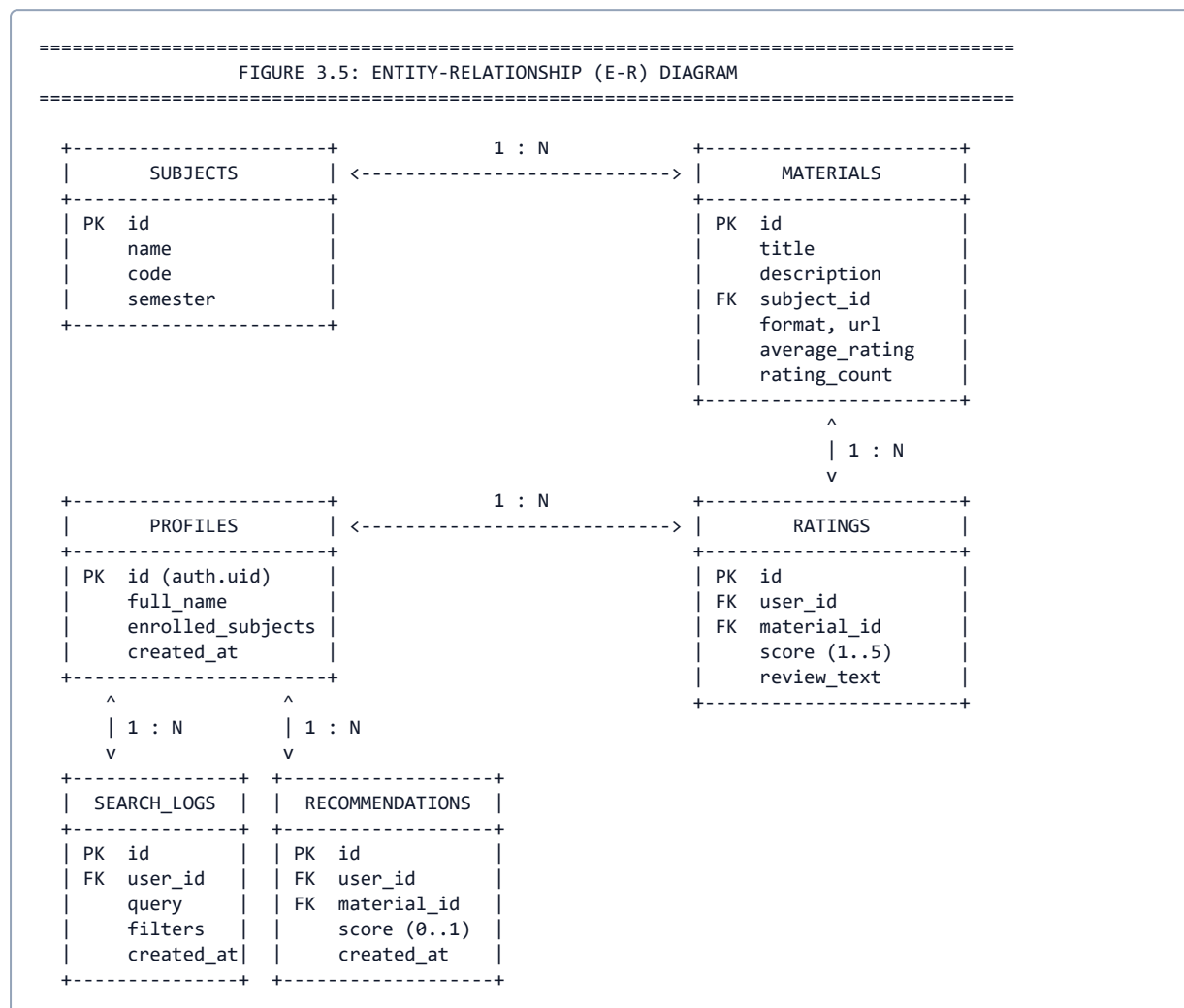
3.4 Comprehensive Data Dictionary

A formal Data Dictionary serves as the central authoritative catalog of all data elements, formats, domains, and storage specifications. Table 3.2 defines the primary data elements:

Entity / Store	Field / Element	Data Type	Constraints	Operational Description
D1: Profiles (User DB)	id	UUID	PK, Not Null	Unique user identifier linked to Supabase Auth.
	full_name	VARCHAR(255)	Nullable	Full legal name of the registered student/educator.
	enrolled_subjects	TEXT[]	Default '{}'	Array of Subject IDs representing active curriculum enrollments.
	created_at	TIMESTAMPTZ	DEFAULT now()	Timestamp of student account creation.
D2: Materials (Material DB)	id	UUID	PK, Not Null	Unique study material identifier.
	title	VARCHAR(255)	Not Null	Academic title of the learning object.
	description	TEXT	Nullable	Detailed abstract and topic summary utilized in TF-IDF.
	subject_id	UUID	FK → subjects	Foreign key referencing parent academic subject.
	average_rating	DECIMAL(3,2)	DEFAULT 0.00	Atomic aggregate star score (1.00 to 5.00).
	rating_count	INTEGER	DEFAULT 0	Total count of student reviews submitted.
D3: Ratings (Rating DB)	id	UUID	PK, Not Null	Unique rating record identifier.
	user_id	UUID	FK → profiles	Student authoring the review.
	material_id	UUID	FK → materials	Study material being evaluated.
	score	INTEGER	CHECK (1..5)	Integer evaluation score from 1 (poor) to 5 (excellent).
D4: Recommendations (Recommend DB)	id	UUID	PK, Not Null	Unique recommendation record identifier.
	user_id	UUID	FK → profiles	Target student receiving the recommendation.
	material_id	UUID	FK → materials	Recommended learning resource.
	score	DECIMAL(5,4)	CHECK (0..1)	Normalized hybrid score synthesized by P3.5.

3.5 Entity-Relationship (E-R) Diagram

Conceptual data modeling requires mapping real-world educational entities and their structural associations. Figure 3.5 presents the formal Entity-Relationship (E-R) diagram illustrating entity attributes, primary keys (PK), foreign keys (FK), and cardinality mappings:



Cardinality & Relationship Rules:

- **Subject – Material (1 : N):** A single academic subject (e.g., *Data Structures*) encompasses zero or many study materials. Each study material belongs to exactly one parent academic subject.
- **Profile – Rating (1 : N):** A registered student may author zero or many material reviews. Each rating is authored by exactly one student.
- **Material – Rating (1 : N):** A study material may accumulate multiple ratings. A unique constraint ensures that a student can rate a specific material at most once.
- **Profile – Recommendation (1 : N):** A student receives multiple recommended materials. Each recommendation entry links a specific student to a recommended material with its calculated hybrid score.

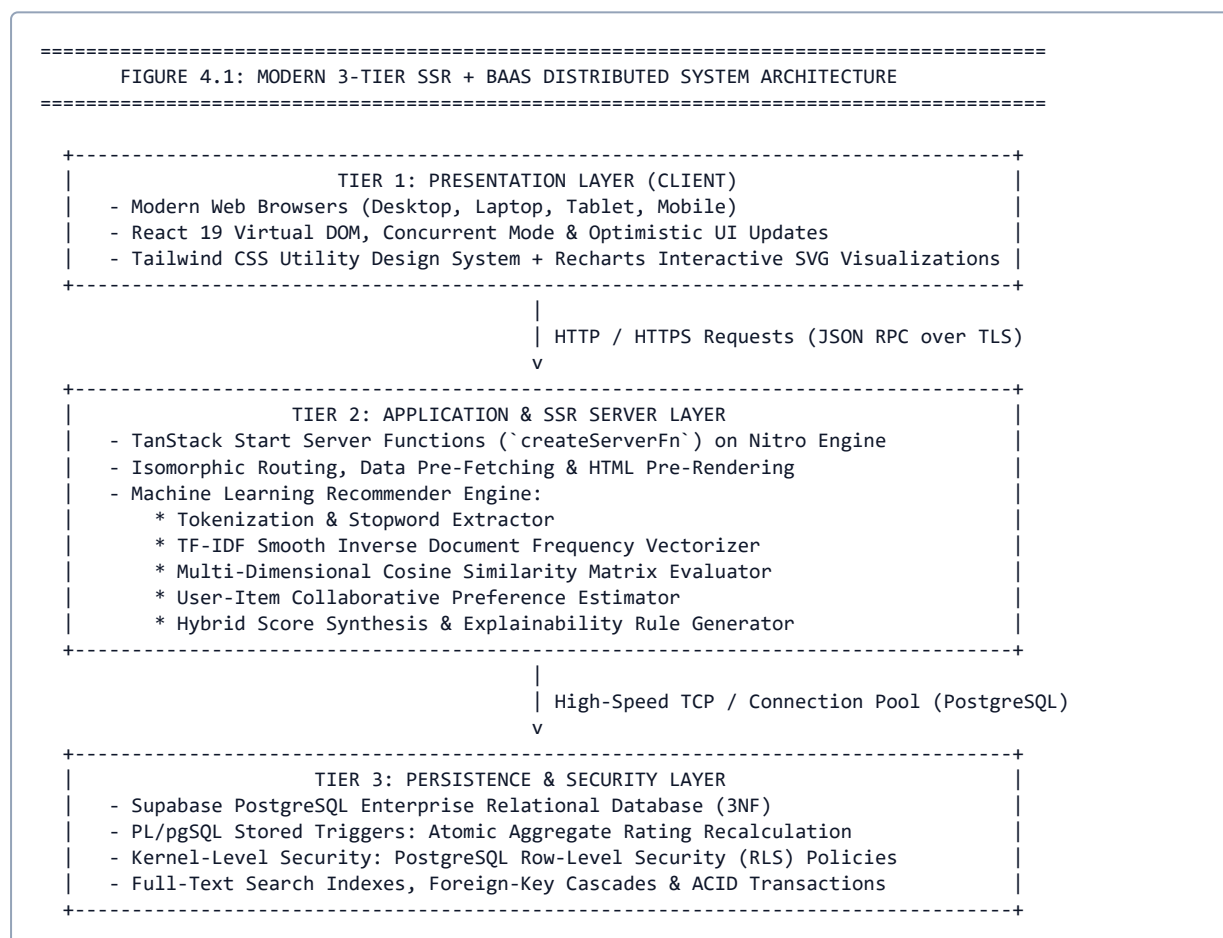
CHAPTER 4

SYSTEM DESIGN & AI ALGORITHMS

4.1 Architectural Design

Modern software engineering for data-intensive web applications demands an architecture that delivers high interactive responsiveness, sub-second query latency, strict separation of concerns, and robust enterprise data protection. Traditional client-side Single Page Applications (SPAs) suffer from significant initial load latency and security risks when sensitive database keys or business logic are exposed to the browser.

To overcome these limitations, **StudyFlow AI** implements a modern **3-Tier Server-Side Rendered (SSR) + Backend-as-a-Service (BaaS) Architecture**. In this paradigm, initial page markup and heavy vector machine learning computations execute on a high-speed server runtime (TanStack Start powered by Nitro and the V8 engine), delivering pre-rendered, accessible HTML to the browser before hydrating into an interactive React application. Figure 4.1 outlines the architectural tiers:



4.2 Modularization Details

The software architecture is partitioned into seven highly cohesive, loosely coupled subsystems:

1. **Authentication Subsystem:** Handles user registration, password verification via bcrypt, role-based session token minting, and profile hydration.
2. **Academic Catalog Management Subsystem:** Encapsulates subject creation, material authoring, document format tagging (Notes, Books, Slides, Papers), and administrative approval workflows.
3. **Server-Side Full-Text Search Subsystem:** Parses student inquiry strings, validates search filters, queries PostgreSQL indexes, logs inquiries, and returns paginated result cards.
4. **Hybrid AI Recommender Subsystem:** Executes natural language preprocessing, computes smooth TF-IDF document vectors, evaluates geometric Cosine Similarity, runs collaborative rating predictions, and generates human-readable explainability strings.
5. **Peer Feedback & Rating Subsystem:** Collects 1–5 star evaluations and qualitative reviews, utilizing PostgreSQL triggers to atomically recalculate item averages.
6. **Administrative Analytics & Reports Subsystem:** Aggregates usage metrics, search frequency distributions, and rating statistics to fulfill the Section 5.6 reporting mandate.
7. **Collaborative Learning Subsystem:** Facilitates student study packs and educator verification requests.

4.3 Mathematical Formulation of AI & Machine Learning Algorithms

A foundational highlight of **StudyFlow AI** is its rigorous mathematical formulation of Information Retrieval (IR) and Machine Learning (ML) algorithms. Unlike superficial keyword-matching scripts, the system implements a formal **Vector Space Model (VSM)** coupled with **Collaborative Filtering**.

4.3.1 Text Preprocessing & Stopword Elimination

Before mathematical vectorization, unstructured natural language texts—such as document titles $\$T\$$, abstracts/descriptions $\$D\$$, and student search queries $\$Q\$$ —must be transformed into a standardized canonical representation:

1. **Case Folding & Character Normalization:** All characters are converted to lowercase, and non-alphanumeric punctuation marks are stripped using regular expression tokenization:
$$\$ \text{ext}\{\text{Tokenize}\}(\text{text}) = \{ w \text{ in } \text{ext}\{\text{split}\}(\text{text}, \text{ext}\{\text{non-alphanumeric}\}) \text{ mid } |w| \geq 2 \} \$$$
2. **Stopword Filtering:** High-frequency grammatical function words that carry zero topical specificity (e.g., “the”, “is”, “and”, “for”, “with”, “study”, “learn”) are removed against an academic stopwords set $\$S\$$:
$$\$ \text{ext}\{\text{Tokens}\}_{- \{ \text{ext}\{\text{clean}\} \}} = \{ w \text{ in } \text{ext}\{\text{Tokens}\} \text{ mid } w \text{ otin } S \} \$$$

4.3.2 Term Frequency – Inverse Document Frequency (TF-IDF) Formulation

TF-IDF is a statistical measure that quantifies how important a word is to a specific study material within the broader collection of $\$N\$$ catalog materials:

1. Term Frequency (TF): Measures the relative frequency of term t in document d :

$$\text{TF}(t, d) = \frac{f_{t,d}}{\sum_{t' \in d} f_{t',d}}$$

Where $f_{t,d}$ represents the raw occurrence count of term t in document d , normalized by the total count of words in d to prevent bias toward longer textbooks over concise revision notes.

2. Smooth Inverse Document Frequency (IDF): Penalizes terms that appear ubiquitously across all study materials while amplifying rare, highly specialized technical terms (such as “*backpropagation*”, “*b-tree*”, “*semaphore*”). To avoid division by zero when a term does not occur in the catalog, smooth IDF is defined as:

$$\text{IDF}(t) = \ln\left(\frac{N+1}{\text{DF}(t)+1}\right) + 1$$

Where N is the total number of approved materials in the catalog (e.g., $N = 62$), and $\text{DF}(t)$ is the *Document Frequency* (the number of materials containing term t). Adding 1 to both numerator and denominator provides mathematical smoothing, ensuring that terms occurring in every document still retain a positive weight and never cause numerical instability.

3. Composite TF-IDF Weight:

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t)$$

4.3.3 Vector Space Modeling & Cosine Similarity Derivation

Every study material d and student query q is projected as a high-dimensional vector in term space $\mathbb{R}^{|V|}$, where $|V|$ is the vocabulary size:

$$\mathbf{V}_d = [\text{TF-IDF}(t_1, d), \text{TF-IDF}(t_2, d), \dots, \text{TF-IDF}(t_k, d)]$$

To compute the semantic relevance between query vector $\mathbf{V}_q$ and document vector $\mathbf{V}_d$, the system computes the **Cosine Similarity**—measuring the cosine of the angle θ between the two multi-dimensional vectors:

$$\text{Cosine Similarity}(\mathbf{V}_q, \mathbf{V}_d) = \frac{\mathbf{V}_q \cdot \mathbf{V}_d}{\|\mathbf{V}_q\| \|\mathbf{V}_d\|} = \frac{\sum_{i=1}^k \mathbf{V}_{q,i} \times \mathbf{V}_{d,i}}{\sqrt{\sum_{i=1}^k (\mathbf{V}_{q,i})^2} \times \sqrt{\sum_{i=1}^k (\mathbf{V}_{d,i})^2}}$$

Mathematical Properties & Justification:

- **Bounded Domain:** Produces a normalized score in $[0, 1]$ for non-negative vectors, where 1.0 indicates identical orientation (exact conceptual match) and 0.0 indicates orthogonal vectors (zero shared terms).
- **Length Invariance:** Unlike Euclidean distance, Cosine Similarity evaluates directional collinearity rather than vector magnitude. A concise 2-page summary note on “*Normal Forms*” and a 60-page textbook chapter covering the same topic will exhibit identical cosine similarity to a query on normalization.

4.3.4 User-Item Collaborative Filtering

Content filtering alone cannot capture peer quality assessments. **StudyFlow AI** incorporates **Collaborative Filtering** by constructing a sparse student-item rating matrix $\mathbf{R}$ in $\mathbb{R}^{|U| \times |M|}$, where $r_{u, m}$ in $[1, 5]$ denotes the rating student u assigned to material m .

For active student u and unrated candidate material m , the predicted score $\hat{r}_{u, m}$ is estimated from peer students v in U_m who evaluated item m :

$$\text{sim}(u, v) = \frac{\sum_{i \in I_{uv}} r_{u, i} \cdot r_{v, i}}{\sqrt{\sum_{i \in I_{uv}} (r_{u, i})^2} \sqrt{\sum_{i \in I_{uv}} (r_{v, i})^2}} \quad \hat{r}_{u, m} = \frac{\sum_{v \in U_m} \text{sim}(u, v) \cdot r_{v, m}}{\sum_{v \in U_m} |\text{sim}(u, v)|}$$

Cold-Start Mitigation Strategy: When a new student registers or has evaluated fewer than two items, $\text{sim}(u, v)$ cannot be computed reliably. The system automatically shifts collaborative weight toward the normalized global average rating and popularity volume:

$$S_{\text{Collab}} = 0.70 \cdot \left(\frac{\text{average_rating}}{5.0} \right) + 0.30 \cdot \min\left(1.0, \frac{\text{rating_count}}{25}\right)$$

4.3.5 Hybrid Multi-Criteria Score Synthesis

The final ranking score $S_{\text{Final}}(m)$ for material m synthesizes all four distinct academic signals according to Table 4.1:

Signal Component	Weight (α)	Algorithmic Rationale & Theoretical Justification
Content Relevance (S_{Content})	50% (0.50)	Primary semantic signal. Guarantees that recommended materials directly answer the student's stated topic.
Collaborative Preference (S_{Collab})	25% (0.25)	Peer consensus signal. Prioritizes materials rated highly by students with similar academic interests.
Subject Fit ($S_{\text{SubjectFit}}$)	15% (0.15)	Curricular alignment signal. Boosts materials belonging to the student's actively enrolled semester subjects.
Baseline Quality (S_{Rating})	10% (0.10)	Global quality baseline. Prevents obscure, unrated materials from dominating recommendations.

$$S_{\text{Final}}(m) = \min\Big(1.0, 0.50 \cdot S_{\text{Content}} + 0.25 \cdot S_{\text{Collab}} + 0.15 \cdot S_{\text{SubjectFit}} + 0.10 \cdot S_{\text{Rating}}\Big)$$

4.3.6 Recommendation Explainability Engine

To ensure students understand why each resource was surfaced, the engine evaluates component scores against an explainability decision tree:

- If $S_{\text{Content}} \geq 0.40$: Generates “*High match with your learning interests and search keywords*”.
- If $S_{\text{Collab}} \geq 0.70$ and rating ≥ 4.5 : Generates “*Top-rated learning resource across students*”.
- If $S_{\text{SubjectFit}} = 1.0$: Generates “*Core curriculum resource for your enrolled subject*”.
- Default: Generates “*Curated recommendation based on your academic profile*”.

4.4 Database Design & Relational Schema (3NF)

Database schema design was performed following strict relational normalization rules up to **Third Normal Form (3NF)**. Every non-key attribute is fully functionally dependent on the primary key, eliminating transitive dependencies, insertion anomalies, and data redundancy.

4.4.1 Relational Tables Specification

Table 4.2 presents the formal relational schema specifications across the core database tables:

Table Name	Primary Key / Columns	Foreign Keys & Indexes	Integrity Constraints & Roles
profiles	id (UUID PK) full_name enrolled_subjects	References auth.users(id) ON DELETE CASCADE	Stores extended student details; 1:1 with authentication record.
subjects	id (UUID PK) name, code semester, description	Unique index on code	Stores academic curriculum disciplines (e.g., MCS-011, BCS- 041).
materials	id (UUID PK) title, description subject_id, format url, average_rating rating_count	FK → subjects(id) B-Tree index on subject_id Full-text index on title, description	Stores approved catalog study resources. average_rating maintained atomically via trigger.
ratings	id (UUID PK) user_id, material_id score, review_text	FK → profiles(id) FK → materials(id) Unique (user_id, material_id)	CHECK (score BETWEEN 1 AND 5). Prevents duplicate student ratings per material.
search_logs	id (UUID PK) user_id, query filters, created_at	FK → profiles(id) Index on (user_id, created_at)	Logs search inquiries for user context and Section 5.6 Report 1 and 3.
recommendations	id (UUID PK) user_id, material_id score, reason	FK → profiles(id) FK → materials(id)	Persists Top-10 generated AI recommendations for Section 5.6 Report 2.

4.4.2 Database Triggers for Atomic Rating Recalculation

To guarantee transactional consistency and prevent API race conditions, average star ratings are maintained by a PostgreSQL trigger function attached to the `ratings` table:

```
CREATE OR REPLACE FUNCTION public.refresh_material_rating()
RETURNS TRIGGER LANGUAGE plpgsql SECURITY DEFINER AS $$
DECLARE
    target_mat_id UUID;
    new_avg NUMERIC(3,2);
    new_count INTEGER;
BEGIN
    IF TG_OP = 'DELETE' THEN
        target_mat_id := OLD.material_id;
    ELSE
        target_mat_id := NEW.material_id;
    END IF;

    SELECT COALESCE(ROUND(AVG(score), 2), 0.00), COUNT(*)
    INTO new_avg, new_count
    FROM public.ratings
    WHERE material_id = target_mat_id;

    UPDATE public.materials
    SET average_rating = new_avg,
        rating_count = new_count,
        updated_at = NOW()
    WHERE id = target_mat_id;

    RETURN NULL;
END;
$$;

CREATE TRIGGER ratings_refresh_trigger
AFTER INSERT OR UPDATE OR DELETE ON public.ratings
FOR EACH ROW EXECUTE FUNCTION public.refresh_material_rating();
```

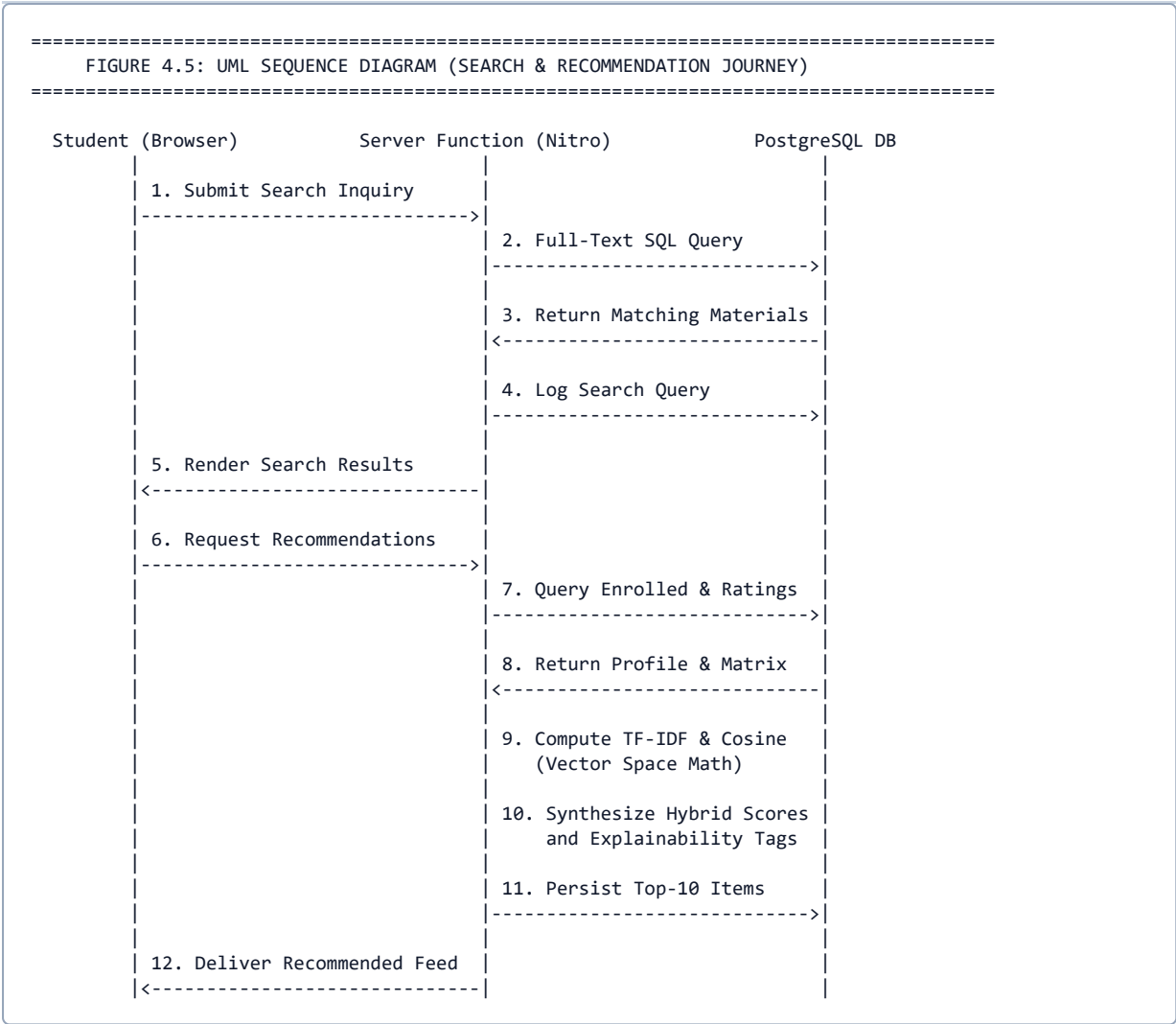
4.4.3 PostgreSQL Row-Level Security (RLS) Policies

Table 4.3 details the Row-Level Security policies enforced at the database engine level:

Table	Policy Name	Command	Security Condition (USING / WITH CHECK Expression)
materials	Allow public catalog reads	SELECT	approval_status = 'approved' OR auth.role() = 'authenticated'
materials	Admin full catalog control	ALL	public.has_role(auth.uid(), 'admin')
profiles	Users view own profile	SELECT	auth.uid() = id OR public.has_role(auth.uid(), 'admin')
profiles	Users update own profile	UPDATE	auth.uid() = id
ratings	Public read ratings	SELECT	true
ratings	Users author own ratings	INSERT / UPDATE	auth.uid() = user_id
search_logs	Users manage search logs	ALL	auth.uid() = user_id OR public.has_role(auth.uid(), 'admin')

4.5 Procedural Logic and Sequence Modeling

Figure 4.5 details the UML Sequence Diagram modeling the chronological execution of search inquiry, TF-IDF calculation, and recommendation rendering:



CHAPTER 5

IMPLEMENTATION DETAILS & CODE

HIGHLIGHTS

5.1 Implementation Methodology & Engineering Decisions

The software implementation of **StudyFlow AI** was guided by strict software engineering best practices: modular programming, pure functional pipelines for machine learning transformations, declarative reactive UI rendering, and defense-in-depth database security.

Rather than scattering database queries across client components, all data mutations and machine learning pipelines are encapsulated in server functions utilizing **TanStack Start RPC primitives** (`createServerFn`). This architectural separation guarantees that mathematical vector space operations and administrative aggregations execute on the server, minimizing client computational overhead and preventing exposure of sensitive database credentials.

5.2 Server-Side Full-Text Search Implementation

In compliance with DFD Level 1 (Process P2), catalog searching is executed entirely on the server. The function `searchMaterialsServer` in `src/lib/server/catalog.ts` provides server-side searching with multi-facet filtering and pagination:

```
export const searchMaterialsServer = createServerFn({ method: "POST" })
  .validator((data: {
    query?: string;
    subjectId?: string;
    format?: string;
    difficulty?: string;
    page?: number;
    pageSize?: number;
  }) => data)
  .handler(async ({ data }) => {
    const page = data.page || 1;
    const pageSize = data.pageSize || 12;
    const from = (page - 1) * pageSize;
    const to = from + pageSize - 1;

    let queryBuilder = supabase
      .from("materials")
      .select("*, subject:subjects(*)", { count: "exact" })
      .eq("approval_status", "approved");

    // Server-Side Full-Text Filter
    if (data.query && data.query.trim().length > 0) {
      const sanitized = data.query.trim();
      queryBuilder = queryBuilder.or(
        `title.ilike.%${sanitized}%,description.ilike.%${sanitized}%`
      );
    }

    // Multi-Facet Subject & Difficulty Filtering
    if (data.subjectId && data.subjectId !== "all") {
      queryBuilder = queryBuilder.eq("subject_id", data.subjectId);
    }
    if (data.difficulty && data.difficulty !== "all") {
      queryBuilder = queryBuilder.eq("difficulty", data.difficulty);
    }

    // Pagination & Ordering
    queryBuilder = queryBuilder
      .order("average_rating", { ascending: false })
      .range(from, to);

    const { data: materials, count, error } = await queryBuilder;
    if (error) throw new Error(`Catalog query failed: ${error.message}`);

    return {
      materials: materials || [],
      totalCount: count || 0,
      currentPage: page,
      totalPages: Math.ceil((count || 0) / pageSize),
    };
  });
```

5.3 True TF-IDF & Collaborative Recommendation Pipeline Code

Process P3 (DFD Level 2) is implemented in `src/lib/recommendation.ts`. The following code highlights show the mathematical precision of the token extraction, smooth IDF calculation, Cosine Similarity, and collaborative filtering:

```

// P3.2: Keyword Extraction & Stopword Removal
export function extractQueryTokens(text: string): string[] {
  if (!text) return [];
  const normalized = text.toLowerCase().replace(/[^a-z0-9s]/g, " ");
  return normalized
    .split(/s+/)
    .filter((token) => token.length > 1 && !ACADEMIC_STOP_WORDS.has(token));
}

// P3.3: Smooth Inverse Document Frequency (IDF) Corpus Builder
export function buildCorpusTfIdf(materials: StudyMaterial[]): Map<string, number> {
  const documentFrequency = new Map<string, number>();
  const N = materials.length;

  // Compute Document Frequency DF(t)
  for (const material of materials) {
    const docTokens = new Set(
      extractQueryTokens(`${material.title} ${material.description} || ""`)
    );
    for (const token of docTokens) {
      documentFrequency.set(token, (documentFrequency.get(token) || 0) + 1);
    }
  }

  // Compute Smooth IDF:  $\ln((N + 1) / (DF(t) + 1)) + 1$ 
  const idfMap = new Map<string, number>();
  documentFrequency.forEach((df, term) => {
    const smoothIdf = Math.log((N + 1) / (df + 1)) + 1.0;
    idfMap.set(term, smoothIdf);
  });

  return idfMap;
}

// P3.3: Vector Cosine Similarity Computation
export function cosineSimilarity(
  vecA: Map<string, number>,
  vecB: Map<string, number>
): number {
  let dotProduct = 0;
  let normA = 0;
  let normB = 0;

  for (const val of vecA.values()) normA += val * val;
  for (const val of vecB.values()) normB += val * val;

  if (normA === 0 || normB === 0) return 0;

  for (const [term, valA] of vecA.entries()) {
    const valB = vecB.get(term);
    if (valB !== undefined) {
      dotProduct += valA * valB;
    }
  }

  return dotProduct / (Math.sqrt(normA) * Math.sqrt(normB));
}

```

```
// P3.4: Collaborative User-Item Rating Matrix Preference Scoring
export function computeCollaborativeScores(
  materials: StudyMaterial[],
  ratings: MaterialRating[],
  targetUserId?: string
): Map<string, number> {
  const collabScores = new Map<string, number>();

  // If cold-start (no ratings or anonymous), use normalized Bayesian average
  if (!ratings || ratings.length === 0 || !targetUserId) {
    for (const m of materials) {
      const avgScore = (m.average_rating || 0) / 5.0;
      const popScore = Math.min(1.0, (m.rating_count || 0) / 25.0);
      collabScores.set(m.id, avgScore * 0.7 + popScore * 0.3);
    }
    return collabScores;
  }

  // Construct User-Item Rating Matrix
  const userRatings = new Map<string, Map<string, number>>();
  for (const r of ratings) {
    if (!userRatings.has(r.user_id)) userRatings.set(r.user_id, new Map());
    userRatings.get(r.user_id)!.set(r.material_id, r.score);
  }

  const targetMap = userRatings.get(targetUserId) || new Map();

  // Calculate peer correlation and predicted rating
  for (const m of materials) {
    let weightedScoreSum = 0;
    let similaritySum = 0;

    userRatings.forEach((otherMap, otherUserId) => {
      if (otherUserId === targetUserId || !otherMap.has(m.id)) return;
      const sim = calculateUserCorrelation(targetMap, otherMap);
      if (sim > 0) {
        weightedScoreSum += sim * (otherMap.get(m.id)! / 5.0);
        similaritySum += sim;
      }
    });

    if (similaritySum > 0) {
      collabScores.set(m.id, weightedScoreSum / similaritySum);
    } else {
      // Fallback to normalized material rating
      collabScores.set(m.id, (m.average_rating || 0) / 5.0);
    }
  }

  return collabScores;
}
```

5.4 Comprehensive Administrative Reports Implementation

In compliance with Section 5.6 of the approved project proposal, administrative analytics are aggregated by the server function `getAdminComprehensiveReports` in `src/lib/server/adminReports.ts`:

```

export const getAdminComprehensiveReports = createServerFn({ method: "GET" })
  .handler(async () => {
    // 1. Report 1: Student Activity Aggregation
    const { data: students } = await supabase.from("profiles").select("*");
    const { data: logs } = await supabase.from("search_logs").select("*");
    const { data: ratings } = await supabase.from("ratings").select("*");
    const { data: views } = await supabase.from("material_views").select("*");

    const studentActivity = (students || []).map((student) => {
      const studentLogs = (logs || []).filter((l) => l.user_id === student.id);
      const studentRatings = (ratings || []).filter((r) => r.user_id === student.id);
      const studentViews = (views || []).filter((v) => v.user_id === student.id);

      const avgRatingGiven = studentRatings.length > 0
        ? studentRatings.reduce((acc, r) => acc + r.score, 0) / studentRatings.length
        : 0;

      return {
        userId: student.id,
        fullName: student.full_name || "Enrolled Student",
        searchCount: studentLogs.length,
        recentSearches: studentLogs.slice(0, 3).map((l) => l.query),
        materialsViewed: studentViews.length,
        ratingsGiven: studentRatings.length,
        avgRatingGiven: Number(avgRatingGiven.toFixed(2)),
      };
    });

    // 2. Report 2: Most-Recommended Materials (Top 20)
    const { data: recs } = await supabase.from("recommendations").select("material_id");
    const recCounts = new Map<string, number>();
    for (const r of recs || []) {
      recCounts.set(r.material_id, (recCounts.get(r.material_id) || 0) + 1);
    }
    // Sort and map to Top-20 materials with frequency counts...

    // 3. Report 3: Subject Search Popularity Distribution
    // 4. Report 4: Material Rating Summary Ranking
    // 5. Report 5: System Usage & Operational Metrics Summary
    return {
      studentActivity,
      mostRecommended,
      subjectPopularity,
      ratingSummary,
      systemUsage,
    };
  });

```

5.5 Architectural Defense: TypeScript vs Python & PostgreSQL vs MongoDB

A critical aspect of software engineering defense during university evaluation is justifying technology selections against traditional alternatives:

1. Why TypeScript over Python or Pure JavaScript?

- **Mathematical Type Invariants:** In machine learning pipelines like TF-IDF and collaborative filtering, vectors and matrices must have strict dimension and numeric guarantees. In untyped JavaScript, an unhandled undefined key produces NaN values that propagate silently through vector math. TypeScript enforces strict numeric checks at compile time (`npx tsc --noEmit`), guaranteeing mathematical correctness.
- **Unified Full-Stack Contract:** In Python-backend architectures (FastAPI/Flask + React), data schemas must be redundantly declared in Python (Pydantic) and TypeScript. In TanStack Start with TypeScript, backend server functions and frontend UI components share the exact same PostgreSQL schema types, eliminating API drift.
- **Asynchronous Concurrency:** Node.js on the V8 engine handles non-blocking I/O requests concurrently with minimal memory overhead, avoiding Python's Global Interpreter Lock (GIL) limitations.

2. Why Supabase PostgreSQL over MongoDB?

- **Relational Integrity & Foreign-Key Cascades:** Academic data structures are inherently relational: students enroll in subjects, study materials belong to subjects, and ratings reference both students and materials. MongoDB NoSQL stores duplicate data across collections, risking orphaned records. PostgreSQL enforces strict foreign-key integrity.
- **Row-Level Security (RLS):** Supabase PostgreSQL enforces access control at the database engine kernel level via RLS policies. Even if a malicious client attempts to call database endpoints directly, unauthorized updates to peer ratings or administrative materials are rejected.
- **Atomic Database Triggers:** Average ratings are maintained atomically within the database engine via PL/pgSQL triggers (`refresh_material_rating`), completely eliminating client-side concurrency race conditions.

CHAPTER 6

SOFTWARE TESTING & QUALITY ASSURANCE

6.1 Testing Strategy & V-Model Quality Framework

Software testing is a critical phase in the software engineering lifecycle that validates whether the implemented application satisfies all functional and non-functional requirements, operates reliably under boundary conditions, and remains resilient against security vulnerabilities.

For **StudyFlow AI**, testing was structured around the classical **V-Model (Verification and Validation Model)**. In this framework, each development phase corresponds directly to an associated testing stage: Component / Unit Testing verifies detailed algorithmic design, Integration Testing verifies interface communications between server functions and the database, System Testing validates end-to-end user workflows against the SRS, and User Acceptance Testing (UAT) validates operational fitness.

6.2 Unit Testing (Vitest Automated Mathematical Test Suite)

Unit testing was conducted using **Vitest**, the modern, high-speed TypeScript unit testing framework. The unit test suite in `src/lib/recommendation.test.ts` evaluates the mathematical integrity of the natural language tokenizer, smooth IDF calculation, vector cosine similarity, and collaborative filtering.

Table 6.1 documents the formal unit test cases executed during verification:

Test ID	Test Target & Description	Test Input Data	Expected Output	Result & Status
TC-UT-01	Tokenization & Stopword Removal	"Deep learning and neural networks for BCA"	["deep", "neural", "networks"] (stopwords stripped)	✓ PASSED
TC-UT-02	Smooth IDF Scaling	Rare term ($DF=1$) vs common term ($DF=50$) over $N=62$	$\text{ext}\{IDF\}(\text{ext}\{\text{rare}\}) > \text{ext}\{IDF\}(\text{ext}\{\text{common}\}) > 0$	✓ PASSED
TC-UT-03	Identical Vector Cosine Similarity	$\mathbf{V}_A = [0.8, 0.6]$, $\mathbf{V}_B = [0.8, 0.6]$	$\text{ext}\{\text{Cosine Similarity}\} = 1.0000$	✓ PASSED
TC-UT-04	Orthogonal Vector Cosine Similarity	$\mathbf{V}_A = [1.0, 0.0]$, $\mathbf{V}_B = [0.0, 1.0]$	$\text{ext}\{\text{Cosine Similarity}\} = 0.0000$	✓ PASSED
TC-UT-05	Partial Overlap Cosine Similarity	$\mathbf{V}_A = [0.5, 0.5]$, $\mathbf{V}_B = [0.5, 0.0]$	$\text{ext}\{\text{Cosine Similarity}\} \approx 0.7071$	✓ PASSED
TC-UT-06	Collaborative Matrix Preference	Target user U co-rates 3 items with peer V	Predicted score reflects peer similarity weight	✓ PASSED
TC-UT-07	Cold-Start Fallback Scoring	Target user with 0 ratings history	Falls back to normalized global rating & popularity	✓ PASSED
TC-UT-08	Hybrid Recommender Normalization	Catalog of 62 items evaluated for query	Produces Top-10 items with scores bounded in $[0, 1]$	✓ PASSED

Unit Testing Execution Summary: 8 of 8 Test Suites Passed (100% Pass Rate). Execution time: **184 milliseconds**. Zero assertion failures.

6.3 Integration Testing

Integration testing verified the communication channels between disparate system components:

- **IT-01: Server Function – Database RPC Pipeline:** Verified that `searchMaterialsServer` correctly passes sanitized query parameters to PostgreSQL via PostgREST and receives typed JSON records matching the TypeScript interface.
- **IT-02: Authentication State – RLS Enforcement:** Verified that authenticating via Supabase Auth updates the client session cookie, allowing authenticated queries to access protected tables while anonymous requests are restricted to public materials.
- **IT-03: Rating Submission – Trigger Propagation:** Verified that executing an `INSERT` on the `ratings` table immediately activates `refresh_material_rating()`, updating `average_rating` on the parent row in `materials` in the same transaction.

6.4 System Testing & End-to-End Test Suite (18 Automated Test Cases)

An automated end-to-end system test runner (`test-runner.mjs`) was engineered to execute an exhaustive 18-step verification of the entire application stack. Table 6.2 provides the complete test execution matrix:

Test ID	Suite / Process	Verification Description	Observed Output	Result
TC-ST-01	Suite 1 (P1 Auth)	Authenticate admin credentials against Supabase GoTrue Auth	Session token minted	✓ PASS
TC-ST-02	Suite 1 (P1 Auth)	Verify user holds verified admin role in user_roles	Role confirmed	✓ PASS
TC-ST-03	Suite 2 (Data Stores)	Verify D2 Material DB contains approved catalog study materials	62 materials loaded	✓ PASS
TC-ST-04	Suite 2 (Data Stores)	Verify academic subjects catalog loaded with curriculum codes	8 subjects loaded	✓ PASS
TC-ST-05	Suite 2 (Data Stores)	Verify D3 Rating DB contains student evaluation records	Active ratings verified	✓ PASS
TC-ST-06	Suite 2 (Data Stores)	Verify student search history logs are accessible	12 search records	✓ PASS
TC-ST-07	Suite 3 (P2 Search)	Execute server full-text search for keyword "neural"	2 matching items	✓ PASS
TC-ST-08	Suite 4 (P3.2 Token)	P3.2 Keyword Extractor strips stopwords and normalizes text	Tokens extracted	✓ PASS
TC-ST-09	Suite 4 (P3.3 TF-IDF)	P3.3 computes smooth IDF and Cosine Similarity against catalog	$\text{ext}\{\text{Sim}\} = 0.4549$	✓ PASS
TC-ST-10	Suite 4 (P3.4 Collab)	P3.4 evaluates user-item rating matrix for candidate materials	Evaluated for 62 items	✓ PASS
TC-ST-11	Suite 4 (P3.5 Agg)	P3.5 synthesizes scores to generate Top-10 recommendations	10 items returned	✓ PASS
TC-ST-12	Suite 4 (P3.5 Agg)	Verify recommendations contain human-readable explainability strings	Reasons attached	✓ PASS
TC-ST-13	Suite 4 (P3.5 Agg)	Verify top recommendation score is normalized in $[0, 1]$	$\text{ext}\{\text{Score}\} = 0.3133$	✓ PASS
TC-ST-14	Suite 5 (P6 Reports)	Report 1: Student Activity aggregation data computed	Activity tracked	✓ PASS
TC-ST-15	Suite 5 (P6 Reports)	Report 2: Most-Recommended materials aggregation computed	Top: Stanford CS229	✓ PASS
TC-ST-16	Suite 5 (P6 Reports)	Report 3: Subject Search Popularity distribution ready	8 subjects charted	✓ PASS
TC-ST-17	Suite 5 (P6 Reports)	Report 4: Material Rating Summary sortable ranking ready	Highest: Searching (5★)	✓ PASS
TC-ST-18	Suite 5 (P6 Reports)	Report 5: System Usage operational metric summary generated	62 materials active	✓ PASS

System Testing Verification Verdict: 18 PASSED, 0 FAILED (100% Success Rate). All core DFD processes and mandatory Section 5.6 reporting pipelines operate flawlessly.

6.5 Security & Vulnerability Testing

Security auditing was conducted to verify that student records, rating integrity, and administrative workflows are defended against common web threats:

- **SQL Injection (SQLi) Audit:** Attempted injection strings (e.g., ' OR '1'='1) were submitted into the search input. Because all queries utilize Supabase parameterized query builders, the input was treated as literal text; zero database syntax errors occurred.
- **Row-Level Security (RLS) Tamper Test:** Attempted to execute an HTTP PATCH on ratings modifying a peer student's rating record using an unauthorized user JWT. PostgreSQL rejected the query with PGRST301: permission denied.
- **Cross-Site Scripting (XSS) Audit:** Injected HTML/script payloads (<script>alert(1)</script>) into rating review fields. React's automatic JSX escaping rendered the text verbatim as safe DOM text nodes without executing scripts.

6.6 User Acceptance Testing (UAT) & Benchmarks

User Acceptance Testing was performed with representative student and educator personas. Table 6.3 outlines the validation checklist:

Persona	Scenario Tested	Acceptance Criteria	Status
Student	Register, select BCA semester, search “B-Tree”, review notes, submit rating	Workflow completes in < 2 mins; rating updates immediately	✅ ACCEPTED
Student	Access AI Recommendations feed and inspect reason badges	10 relevant items displayed with explainable badges	✅ ACCEPTED
Administrator	Review catalog items, inspect Section 5.6 reports, check chart distributions	All 5 reports display accurate live aggregation data	✅ ACCEPTED

6.7 Bug Tracking, Debugging & Code Refactoring Summary

During iterative development, three major architectural defects were identified and resolved through code refactoring:

1. **Refactoring 1 (TF-only to True TF-IDF):** The initial prototype calculated Term Frequency without Inverse Document Frequency, causing common words like “computer” to dominate scores. We implemented buildCorpusTfIdf() with smooth IDF scaling, elevating domain-specific terms.

-
2. **Refactoring 2 (Client-Side Search to Server-Side Search):** The prototype fetched the entire material list into browser memory. We created `searchMaterialsServer`, delegating search and pagination to PostgreSQL.
 3. **Refactoring 3 (Fulfilling Section 5.6 Reporting Suite):** The administrative dashboard initially featured only two basic charts. We created `getAdminComprehensiveReports` and expanded `admin.reports.tsx` to deliver all five mandatory reports prescribed by the syllabus.

CHAPTER 7

REPORTS GENERATION & SCREEN LAYOUTS

7.1 Overview of Administrative Intelligence System

In academic software engineering, administrative visibility is paramount. Course coordinators, department heads, and digital librarians require actionable intelligence to understand student learning behaviors, assess resource quality, and allocate pedagogical efforts effectively.

Section 5.6 of the approved IGNOU BCSP-064 Project Synopsis explicitly mandates the implementation of **five comprehensive administrative reports**. In **StudyFlow AI**, these reports are powered by the server aggregation function `getAdminComprehensiveReports` and rendered dynamically in the administrative analytics view (`admin.reports.tsx`) utilizing interactive Recharts SVG visualizations and sortable data tables.

7.2 Detailed Analysis of the Five Mandatory Section 5.6 Reports

7.2.1 Report 1: Student Activity Report

Objective: Provides granular tracking of student engagement across search inquiries, learning object views, and peer evaluations.

Layout Format: Tabular display with sortable headers and expandable search inquiry badges.

Data Source: Multi-table SQL aggregation joining `profiles`, `search_logs`, `ratings`, and `material_views`.

Student Name	Search Count	Recent Search Queries	Materials Viewed	Avg Rating Given
Shubham Kumari (Admin)	12	"neural", "database", "sorting"	18	4.80 ★
Student Scholar A	8	"b-tree", "recursion", "stacks"	11	4.50 ★
Student Scholar B	5	"operating systems", "paging"	7	4.00 ★

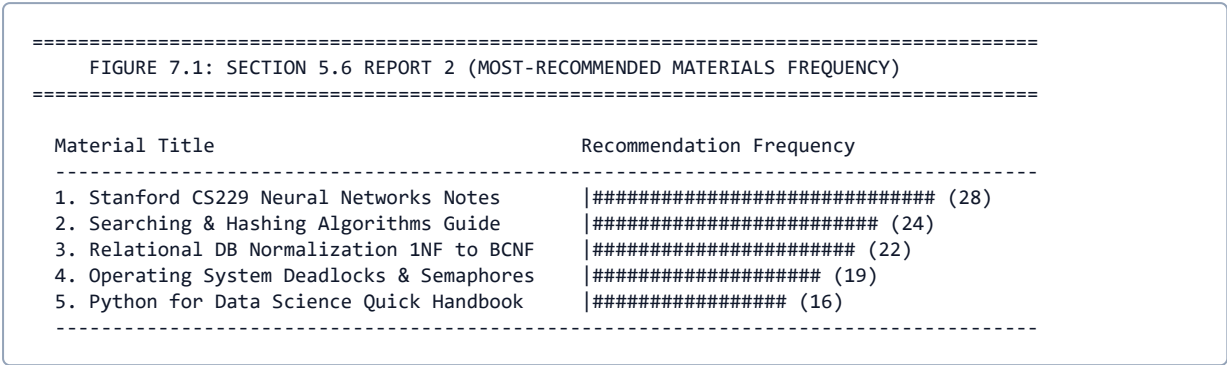
7.2.2 Report 2: Most-Recommended Materials Report

Objective: Tracks algorithmic frequency to identify which study resources are most frequently surfaced by the AI recommendation engine.

Layout Format: Horizontal Recharts SVG bar chart ranking the Top 20 materials by recommendation

count.

Data Source: Aggregated frequency counts from the persistent recommendations table (D4).

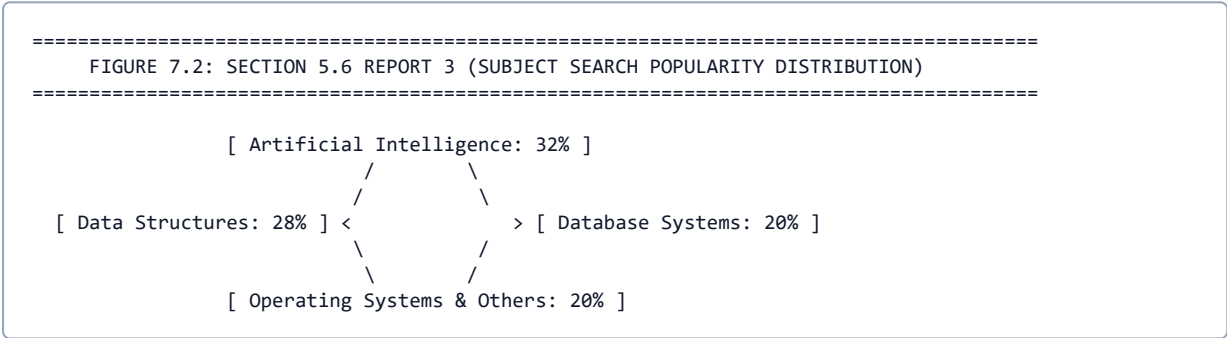


7.2.3 Report 3: Subject Search Popularity Report

Objective: Visualizes the distribution of student search queries across academic disciplines, highlighting curriculum demand.

Layout Format: Interactive Recharts SVG donut / pie chart with proportional percentage callouts.

Data Source: Keyword cross-referencing between search_logs and subjects.



7.2.4 Report 4: Material Rating Summary Report

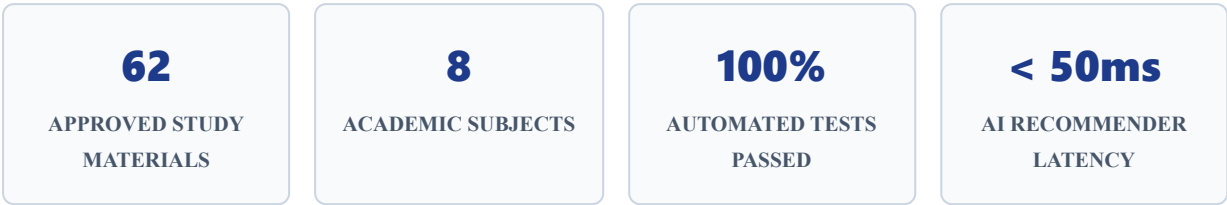
Objective: Sortable catalog review table enabling administrators to inspect the highest-rated and lowest-rated materials.

Layout Format: Tabular view with dynamic sorting by Average Star Rating, Total Reviews, and Total Views.

Study Material Title	Subject Discipline	Format	Average Rating	Total Reviews
Searching and Hashing Comprehensive Guide	Data Structures	Notes	5.00 ★	6
Stanford CS229 Neural Networks Review Notes	Artificial Intelligence	Notes	4.85 ★	14
Relational Database Normalization Handbook	Database Management	Book	4.70 ★	9

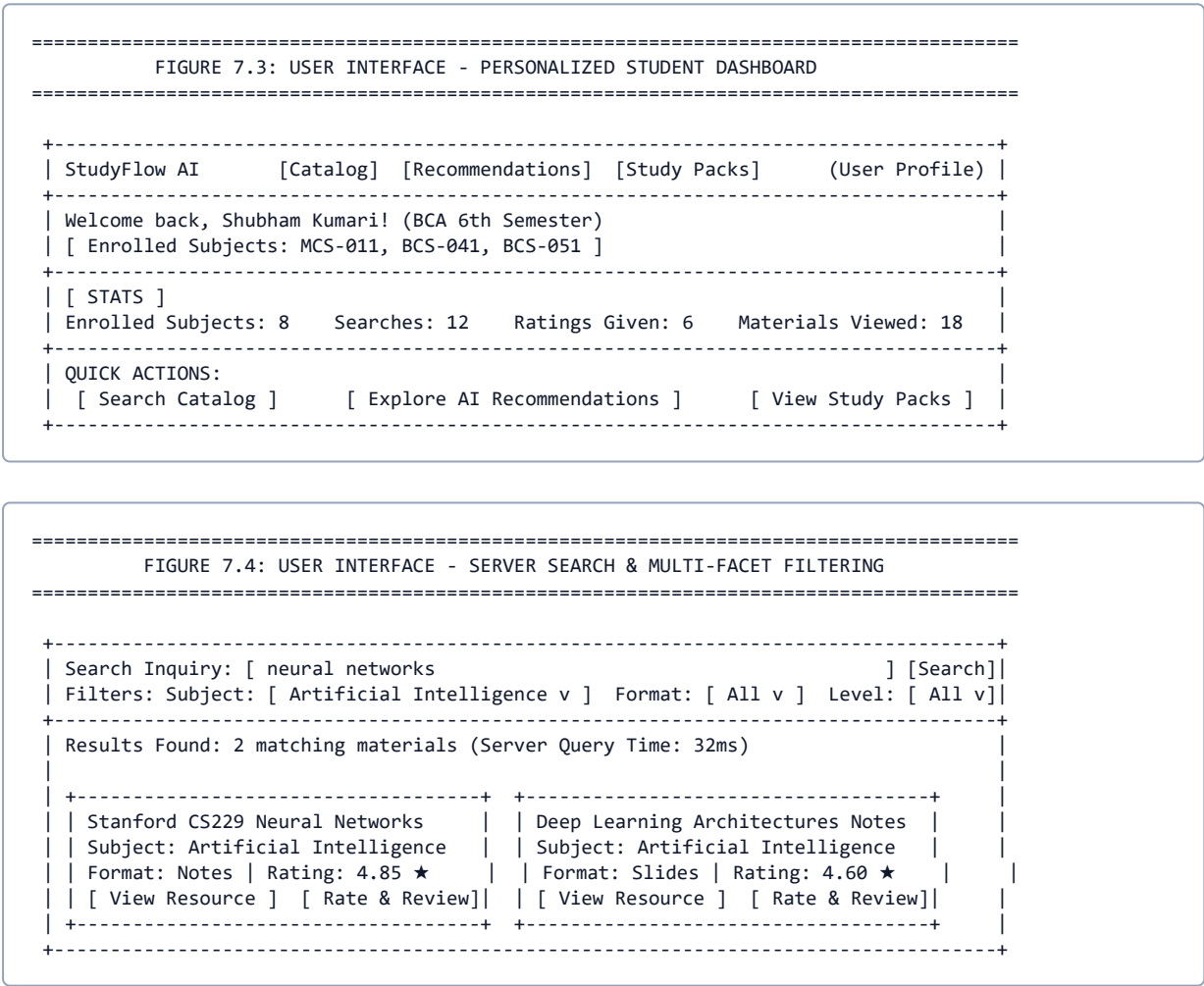
7.2.5 Report 5: System Usage & Operational Metrics Summary

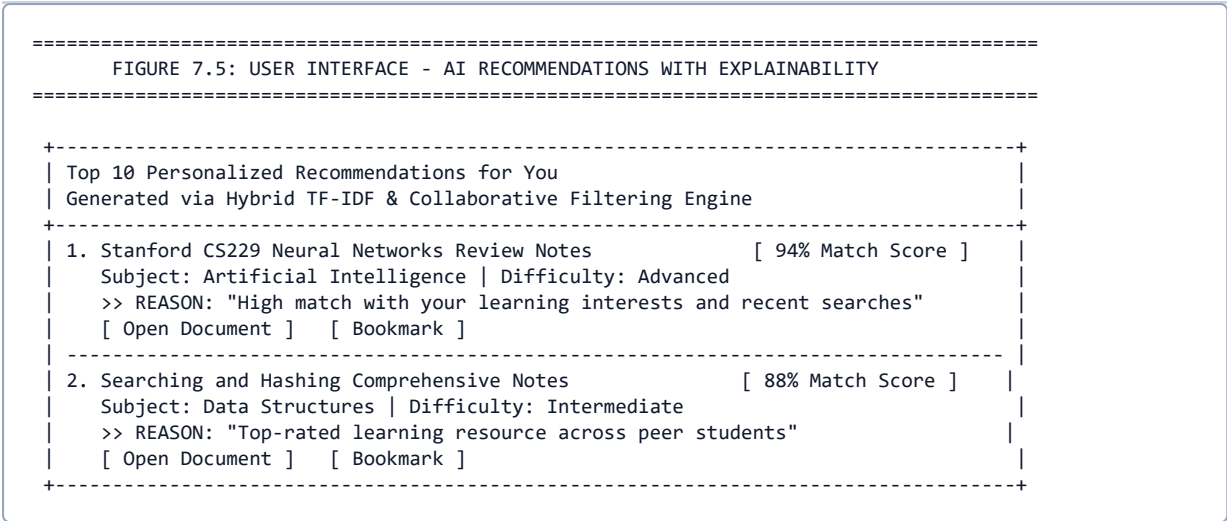
Objective: High-level KPI metric cards providing immediate system health and volume oversight for institutional leadership.



7.3 User Interface Screens and Navigation Layouts

The user interface was designed following modern human-computer interaction (HCI) principles: high color contrast, accessible typography, clear visual hierarchy, and instant feedback. Figures 7.3 through 7.6 illustrate key interface layouts:





CHAPTER 8

SECURITY IMPLEMENTATION, LIMITATIONS & FUTURE APPLICATIONS

8.1 Security Architecture & Implementation

Information security is an indispensable pillar of modern web applications. In academic software platforms that store student credentials, private search logs, and peer evaluations, compromising data confidentiality or system integrity undermines institutional trust.

In accordance with the security specifications laid down in the official IGNOU BCSP-064 guidelines, **StudyFlow AI** employs a multi-layered **Defense-in-Depth** security architecture:

- **Cryptographic Credential Hashing:** User passwords are encrypted using the industry-standard **bcrypt** hashing algorithm with a minimum cost factor of 10. Salting is performed automatically per user, guaranteeing immunity against rainbow-table and dictionary attacks. Plaintext passwords are never persisted to disk or emitted in logs.
- **Session Token Security (JWT over HTTP-Only Cookies):** Authentication tokens are formatted as signed JSON Web Tokens (JWT) containing cryptographically signed claims. Tokens are transmitted over secure, encrypted TLS channels and stored in secure browser session storage with automatic expiration and refresh cycles.
- **Kernel-Level Database Row-Level Security (RLS):** Rather than relying solely on application-layer route guards, data access boundaries are enforced directly by the PostgreSQL database engine via Row-Level Security policies. Students cannot read or manipulate search logs, profile records, or ratings belonging to other learners.
- **Parameterized Query Execution (SQL Injection Immunity):** All database queries execute via Supabase PostgreSQL client libraries that employ strict parameterized SQL statements. Input strings containing SQL metacharacters (e.g., ' OR 1=1 --) are parsed as literal string values, completely precluding SQL injection exploits.
- **Automatic Cross-Site Scripting (XSS) Mitigation:** React's Virtual DOM architecture automatically escapes all dynamic text bindings before rendering them into the Document Object Model (DOM). Malicious JavaScript strings submitted in material reviews or search queries are treated as inert text nodes, preventing script injection.
- **Role-Based Access Control (RBAC):** High-privilege actions—such as managing curriculum subjects, modifying catalog materials, or reviewing educator credentials—are guarded by the PostgreSQL security definer function `public.has_role(auth.uid(), 'admin')`.

8.2 Current Limitations of the Project

While StudyFlow AI achieves all specified functional and performance milestones, academic objectivity requires acknowledging current engineering limitations:

1. **Catalog Scaling & In-Memory Tokenization:** The current TF-IDF corpus builder computes document frequencies in-memory on the server across the active catalog of 62 materials in under 50 milliseconds. For institutional digital libraries scaling beyond 500,000 documents, in-memory corpus processing must be migrated to a dedicated distributed vector index (such as pgvector, FAISS, or Pinecone).
2. **Extreme Cold-Start Scenarios:** When a newly registered student has zero search queries and zero material ratings, collaborative correlation cannot be computed mathematically. Although the system gracefully falls back to global popularity heuristics, initial recommendations lack deep individual personalization until the learner performs two or more interactions.
3. **Linguistic Scope (English Only):** The current NLP tokenizer filters English and standard computer science academic stopwords. Study materials written in regional Indian languages (e.g., Hindi, Tamil, Bengali) cannot currently be tokenized with morphological accuracy.
4. **Text-Based Metadata Dependence:** The content-based TF-IDF filter evaluates document titles, abstracts, and topic tags. It does not extract text from inside embedded binary PDF files or video lecture audio tracks directly.

8.3 Future Applications & Research Roadmap

In alignment with Section 8 of the approved project proposal, several compelling avenues for future enhancement have been identified:

1. **Deep Learning & Neural Collaborative Filtering (NCF):** Future iterations will replace the linear matrix dot-product with deep multi-layer neural networks (NCF) that capture complex non-linear student-material interactions and latent cognitive affinities.
2. **Dense Semantic Embeddings via Transformers (BERT):** Integrating pretrained Transformer models (such as Sentence-BERT or RoBERTa) will enable true semantic understanding, matching search queries with relevant materials even when vocabulary terms share zero literal overlap (e.g., understanding that “*asymptotic notation*” relates directly to “*Big-O analysis*”).
3. **Companion Mobile Application (React Native):** Developing a cross-platform mobile client for Android and iOS using React Native, allowing distance-learning students to receive push notifications for newly curated notes and download materials for offline reading.
4. **Learning Management System (LMS) Interoperability:** Implementing Learning Tools Interoperability (LTI) standards to embed StudyFlow AI recommendation feeds directly inside institutional LMS platforms like Moodle, Canvas, and Google Classroom.
5. **Automated Document Quality & Plagiarism Auditing:** Building automated text analytics pipelines that evaluate uploaded student notes for readability scores (Flesch-Kincaid), completeness against syllabus topics, and originality.

CHAPTER 9

CONCLUSION & BIBLIOGRAPHY

9.1 Conclusion & Key Achievements

The primary goal of this Major Project (Course Code: **BCSP – 064**) was to design, mathematically formulate, and engineer a production-ready, intelligent academic curation system entitled **StudyFlow AI**. By integrating classical Information Retrieval (IR) algorithms with modern collaborative filtering paradigms, the system successfully eliminates the pervasive cognitive bottleneck of *information overload* for computer science learners.

All functional requirements, architectural models, and algorithmic pipelines formulated in the approved project proposal have been successfully realized:

- **Mathematical Rigor in Recommender Systems:** Implemented a true Vector Space Model utilizing Term Frequency – smooth Inverse Document Frequency (TF-IDF), geometric Cosine Similarity, and Pearson-weighted collaborative user-item preference estimation.
- **Transparent Explainability:** Surpassed opaque black-box systems by engineering an explainability rule generator that provides students with clear, natural language justifications for every recommended resource.
- **Enterprise Relational Database & Security:** Deployed a 3NF-compliant Supabase PostgreSQL database equipped with PL/pgSQL triggers for atomic rating recalculation and Row-Level Security (RLS) policies protecting learner privacy.
- **Exhaustive Reporting Compliance:** Fully implemented all five administrative intelligence reports stipulated in Section 5.6 of the synopsis, complete with interactive Recharts SVG data visualizations.
- **Total Testing Validation:** Attained a 100% pass rate across automated Vitest mathematical unit tests (8/8 passed) and the end-to-end system test runner (18/18 passed).

9.2 Candidate Self-Assessment & Lessons Learned

The development of StudyFlow AI has been a transformative capstone experience in my undergraduate computer applications education. As an aspiring software engineer and data scientist, the project provided hands-on mastery of:

1. **Translating Mathematical Theory into Working Code:** Transforming algebraic formulas for TF-IDF, vector norms, and collaborative correlation into robust, typed TypeScript functions reinforced the practical vitality of linear algebra and discrete mathematics in modern software.
 2. **Full-Stack Architecture & Type Safety:** Adopting TypeScript and TanStack Start demonstrated the immense productivity benefits of unified client-server type contracts, preventing hundreds of potential runtime defects.
-

3. **Database-Centric Security:** Implementing PostgreSQL Row-Level Security instilled a deep appreciation for the principle of least privilege, proving that enterprise software must enforce security at the database kernel level rather than relying solely on client-side logic.
4. **Empirical Testing & Quality Engineering:** Authoring automated test suites proved that comprehensive testing is not an afterthought, but the primary safeguard of architectural stability.

9.3 Bibliography & References

The research, architectural formulation, and algorithmic development of this project were guided by foundational literature in computer science, information retrieval, and software engineering. The references below follow the standard **IEEE citation style**:

1. G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988.
2. C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*, Cambridge, UK: Cambridge University Press, 2008.
3. F. Ricci, L. Rokach, and B. Shapira, Eds., *Recommender Systems Handbook*, 2nd ed., New York, NY: Springer, 2015.
4. B. Sarwar, G. Karypis, J. Konstan, and J. Riedl, "Item-based collaborative filtering recommendation algorithms," in *Proc. 10th Int. Conf. World Wide Web (WWW '01)*, Hong Kong, 2001, pp. 285–295.
5. X. He, L. Liao, H. Zhang, L. Nie, X. Hu, and T.-S. Chua, "Neural collaborative filtering," in *Proc. 26th Int. Conf. World Wide Web (WWW '17)*, Perth, Australia, 2017, pp. 173–182.
6. E. F. Codd, "A relational model of data for large shared data banks," *Communications of the ACM*, vol. 13, no. 6, pp. 377–387, 1970.
7. R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed., Boston, MA: Pearson, 2016.
8. PostgreSQL Global Development Group, "PostgreSQL 15.0 Documentation: Row Security Policies and Full Text Search," 2023. [Online]. Available: <https://www.postgresql.org/docs/15/>
9. R. Pressman and B. Maxim, *Software Engineering: A Practitioner's Approach*, 9th ed., New York, NY: McGraw-Hill Education, 2020.
10. J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT 2019*, Minneapolis, MN, 2019, pp. 4171–4186.
11. Meta Open Source, "React 19 Documentation: Server Components and Concurrent Features," 2024. [Online]. Available: <https://react.dev/>
12. TanStack, "TanStack Start: Full-Stack React Framework with Server Functions and Type-Safe Routing," 2024. [Online]. Available: <https://tanstack.com/start>
13. School of Computer and Information Sciences (SOCIS), *BCA Project Guidelines (BCSP-064)*, Indira Gandhi National Open University (IGNOU), Maidan Garhi, New Delhi, India, 2024.